

ESP32 Exterior - Sistema de Fermentacion de Cafe

1.0

Generado por Doxygen 1.17.0

1 Jerarquía de directorios	1
1.1 Directorios	1
2 Índice de clases	3
2.1 Lista de clases	3
3 Índice de archivos	5
3.1 Lista de archivos	5
4 Documentación de directorios	7
4.1 Referencia del directorio main/drivers	7
4.2 Referencia del directorio main/include	7
4.3 Referencia del directorio main	8
4.4 Referencia del directorio main/mqtt	8
4.5 Referencia del directorio main/tasks	8
4.6 Referencia del directorio main/wifi	8
5 Documentación de clases	9
5.1 Referencia de la estructura datos_ambiente_t	9
5.1.1 Descripción detallada	9
5.1.2 Documentación de datos miembro	10
5.1.2.1 anio	10
5.1.2.2 dia	10
5.1.2.3 hora	10
5.1.2.4 humedad	10
5.1.2.5 mes	10
5.1.2.6 minuto	10
5.1.2.7 segundo	10
5.1.2.8 temperatura	11
5.2 Referencia de la estructura dht11_data_t	11
5.2.1 Descripción detallada	11
5.2.2 Documentación de datos miembro	11
5.2.2.1 humedad	11
5.2.2.2 temperatura	11
5.3 Referencia de la estructura rtc_time_t	12
5.3.1 Descripción detallada	12
5.3.2 Documentación de datos miembro	12
5.3.2.1 anio	12
5.3.2.2 dia	12
5.3.2.3 horas	13
5.3.2.4 mes	13
5.3.2.5 minutos	13
5.3.2.6 segundos	13

6 Documentación de archivos	15
6.1 Referencia del archivo main/drivers/dht11.c	15
6.1.1 Descripción detallada	15
6.1.2 Documentación de «define»	16
6.1.2.1 DHT11_GPIO	16
6.1.3 Documentación de funciones	16
6.1.3.1 dht11_read()	16
6.1.3.2 set_input()	17
6.1.3.3 set_output()	18
6.2 Referencia del archivo main/drivers/dht11.h	18
6.2.1 Descripción detallada	18
6.2.2 Documentación de funciones	19
6.2.2.1 dht11_read()	19
6.3 dht11.h	20
6.4 Referencia del archivo main/drivers/ds3231.c	20
6.4.1 Descripción detallada	21
6.4.2 Documentación de «define»	21
6.4.2.1 DS3231_ADDR	21
6.4.2.2 I2C_PORT	21
6.4.2.3 TAG	22
6.4.3 Arquitectura del sistema	22
6.4.4 Documentación de funciones	22
6.4.4.1 bcd_to_dec()	22
6.4.4.2 dec_to_bcd()	23
6.4.4.3 ds3231_clear_alarm_flag()	23
6.4.4.4 ds3231_read_time()	24
6.4.4.5 ds3231_set_alarm_every_minute()	25
6.4.4.6 ds3231_set_time()	26
6.5 Referencia del archivo main/drivers/ds3231.h	27
6.5.1 Descripción detallada	27
6.5.2 Arquitectura del sistema	28
6.5.3 Documentación de funciones	28
6.5.3.1 ds3231_clear_alarm_flag()	28
6.5.3.2 ds3231_read_time()	29
6.5.3.3 ds3231_set_alarm_every_minute()	30
6.5.3.4 ds3231_set_time()	31
6.6 ds3231.h	32
6.7 Referencia del archivo main/drivers/lcd_parallel.c	33
6.7.1 Descripción detallada	33
6.7.2 Rol dentro del sistema	34
6.7.3 Documentación de «define»	34
6.7.3.1 LCD_D4	34

6.7.3.2 LCD_D5	34
6.7.3.3 LCD_D6	35
6.7.3.4 LCD_D7	35
6.7.3.5 LCD_E	35
6.7.3.6 LCD_RS	35
6.7.4 Documentación de funciones	35
6.7.4.1 lcd_clear()	35
6.7.4.2 lcd_cmd()	36
6.7.4.3 lcd_data()	36
6.7.4.4 lcd_init()	36
6.7.4.5 lcd_print()	37
6.7.4.6 lcd_send_byte()	37
6.7.4.7 lcd_send_nibble()	38
6.7.4.8 lcd_set_cursor()	38
6.8 Referencia del archivo main/drivers/lcd_parallel.h	39
6.8.1 Descripción detallada	39
6.8.2 Integración en el sistema	39
6.8.3 Documentación de funciones	40
6.8.3.1 lcd_clear()	40
6.8.3.2 lcd_init()	40
6.8.3.3 lcd_print()	41
6.8.3.4 lcd_set_cursor()	41
6.9 lcd_parallel.h	42
6.10 Referencia del archivo main/drivers/sd_card.c	42
6.10.1 Descripción detallada	43
6.10.2 Rol dentro del sistema	43
6.10.3 Estrategia Store-and-Forward	44
6.10.4 Documentación de «define»	44
6.10.4.1 PIN_NUM_CLK	44
6.10.4.2 PIN_NUM_CS	44
6.10.4.3 PIN_NUM_MISO	44
6.10.4.4 PIN_NUM_MOSI	45
6.10.4.5 SD_PENDING_MQTT_PATH	45
6.10.4.6 SD_SPI_MAX_FREQ_KHZ	45
6.10.5 Documentación de funciones	45
6.10.5.1 sd_append_pending_mqtt()	45
6.10.5.2 sd_close_pending_mqtt()	46
6.10.5.3 sd_delete_pending_mqtt()	46
6.10.5.4 sd_init()	47
6.10.5.5 sd_open_pending_mqtt_read()	47
6.10.5.6 sd_pending_mqtt_exists()	48
6.10.5.7 sd_read_pending_mqtt_line()	48

6.10.5.8 sd_write_line()	49
6.11 Referencia del archivo main/drivers/sd_card.h	50
6.11.1 Descripción detallada	50
6.11.2 Integración en el sistema	51
6.11.3 Documentación de funciones	51
6.11.3.1 sd_append_pending_mqtt()	51
6.11.3.2 sd_close_pending_mqtt()	52
6.11.3.3 sd_delete_pending_mqtt()	52
6.11.3.4 sd_init()	53
6.11.3.5 sd_open_pending_mqtt_read()	54
6.11.3.6 sd_pending_mqtt_exists()	54
6.11.3.7 sd_read_pending_mqtt_line()	55
6.11.3.8 sd_write_line()	55
6.12 sd_card.h	56
6.13 Referencia del archivo main/include/common.h	56
6.13.1 Descripción detallada	57
6.14 common.h	57
6.15 Referencia del archivo main/include/queues.h	57
6.15.1 Descripción detallada	58
6.15.2 Arquitectura de comunicación	58
6.15.3 Documentación de variables	58
6.15.3.1 cola_datos	58
6.15.3.2 cola_display	59
6.15.3.3 cola_mqtt	59
6.16 queues.h	59
6.17 Referencia del archivo main/main.c	59
6.17.1 Descripción detallada	60
6.17.2 Documentación de «define»	62
6.17.2.1 I2C_MASTER_FREQ_HZ	62
6.17.2.2 I2C_MASTER_SCL_IO	62
6.17.2.3 I2C_MASTER_SDA_IO	62
6.17.2.4 RTC_INT_GPIO	62
6.17.2.5 WIFI_CONNECT_TIMEOUT_MS	62
6.17.3 Documentación de funciones	62
6.17.3.1 app_main()	62
6.17.3.2 i2c_init()	64
6.17.3.3 task_display()	64
6.17.3.4 task_logger()	64
6.17.3.5 task_mqtt()	64
6.17.3.6 task_sensores()	65
6.17.4 Documentación de variables	65
6.17.4.1 cola_datos	65

6.17.4.2 cola_display	65
6.17.4.3 cola_mqtt	66
6.17.4.4 main_task_handle	66
6.18 Referencia del archivo main/mqtt/mqtt_manager.c	66
6.18.1 Descripción detallada	67
6.18.2 Documentación de «define»	67
6.18.2.1 MQTT_BROKER_URI	67
6.18.2.2 MQTT_CONNECTED_BIT	68
6.18.2.3 MQTT_ERROR_BIT	68
6.18.2.4 MQTT_PUBLISHED_BIT	68
6.18.3 Documentación de funciones	68
6.18.3.1 mqtt_event_handler()	68
6.18.3.2 mqtt_manager_start()	69
6.18.3.3 mqtt_manager_stop()	69
6.18.3.4 mqtt_publish()	69
6.18.3.5 mqtt_wait_connected()	70
6.18.4 Documentación de variables	71
6.18.4.1 mqtt_client	71
6.18.4.2 mqtt_event_group	71
6.18.4.3 TAG	71
6.19 Referencia del archivo main/mqtt/mqtt_manager.h	71
6.19.1 Descripción detallada	71
6.19.2 Documentación de funciones	72
6.19.2.1 mqtt_manager_start()	72
6.19.2.2 mqtt_manager_stop()	72
6.19.2.3 mqtt_publish()	72
6.19.2.4 mqtt_wait_connected()	73
6.20 mqtt_manager.h	74
6.21 Referencia del archivo main/tasks/task_display.c	74
6.21.1 Descripción detallada	74
6.21.2 Documentación de funciones	75
6.21.2.1 task_display()	75
6.21.3 Documentación de variables	76
6.21.3.1 main_task_handle	76
6.22 Referencia del archivo main/tasks/task_logger.c	76
6.22.1 Descripción detallada	76
6.22.2 Documentación de funciones	77
6.22.2.1 task_logger()	77
6.23 Referencia del archivo main/tasks/task_mqtt.c	77
6.23.1 Descripción detallada	78
6.23.2 Documentación de «define»	79
6.23.2.1 MQTT_CONNECT_TIMEOUT_MS	79

6.23.2.2 MQTT_PENDING_PAYLOAD_SIZE	79
6.23.2.3 MQTT_PUBLISH_TIMEOUT_MS	79
6.23.2.4 MQTT_TOPIC	79
6.23.3 Documentación de funciones	79
6.23.3.1 resend_pending_mqtt_messages()	79
6.23.3.2 save_pending_payload()	80
6.23.3.3 task_mqtt()	80
6.23.4 Documentación de variables	81
6.23.4.1 main_task_handle	81
6.24 Referencia del archivo main/tasks/task_sensores.c	81
6.24.1 Descripción detallada	81
6.24.2 Documentación de funciones	83
6.24.2.1 task_sensores()	83
6.25 Referencia del archivo main/wifi/wifi_manager.c	83
6.25.1 Descripción detallada	84
6.25.2 Documentación de «define»	84
6.25.2.1 WIFI_CONNECTED_BIT	84
6.25.2.2 WIFI_FAIL_BIT	85
6.25.2.3 WIFI_MAX_RETRIES	85
6.25.2.4 WIFI_PASS	85
6.25.2.5 WIFI_SCAN_MAX_AP	85
6.25.2.6 WIFI_SSID	85
6.25.3 Documentación de funciones	85
6.25.3.1 auth_mode_name()	85
6.25.3.2 wifi_event_handler()	86
6.25.3.3 wifi_init()	86
6.25.3.4 wifi_scan_print()	87
6.25.3.5 wifi_wait_connected()	88
6.25.4 Documentación de variables	88
6.25.4.1 retry_count	88
6.25.4.2 TAG	89
6.25.4.3 wifi_event_group	89
6.26 Referencia del archivo main/wifi/wifi_manager.h	89
6.26.1 Descripción detallada	89
6.26.2 Documentación de funciones	90
6.26.2.1 wifi_init()	90
6.26.2.2 wifi_scan_print()	91
6.26.2.3 wifi_wait_connected()	91
6.27 wifi_manager.h	92
Índice alfabético	93

Capítulo 1

Jerarquía de directorios

1.1. Directorios

drivers	7
dht11.c	15
dht11.h	18
ds3231.c	20
ds3231.h	27
lcd_parallel.c	33
lcd_parallel.h	39
sd_card.c	42
sd_card.h	50
include	7
common.h	56
queues.h	57
main	8
drivers	7
dht11.c	15
dht11.h	18
ds3231.c	20
ds3231.h	27
lcd_parallel.c	33
lcd_parallel.h	39
sd_card.c	42
sd_card.h	50
include	7
common.h	56
queues.h	57
mqtt	8
mqtt_manager.c	66
mqtt_manager.h	71
tasks	8
task_display.c	74
task_logger.c	76
task_mqtt.c	77
task_sensores.c	81
wifi	8
wifi_manager.c	83
wifi_manager.h	89
main.c	59

mqtt	8
mqtt_manager.c	66
mqtt_manager.h	71
tasks	8
task_display.c	74
task_logger.c	76
task_mqtt.c	77
task_sensores.c	81
wifi	8
wifi_manager.c	83
wifi_manager.h	89

Capítulo 2

Índice de clases

2.1. Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

datos_ambiente_t	Estructura que almacena una medición ambiental completa	9
dht11_data_t	Estructura que almacena las mediciones obtenidas del sensor DHT11	11
rtc_time_t	Representa una fecha y hora completas	12

Capítulo 3

Índice de archivos

3.1. Lista de archivos

Lista de todos los archivos con breves descripciones:

main/main.c	
Punto de entrada del sistema ESP32 para monitoreo ambiental	59
main/drivers/dht11.c	
Implementación del driver para sensor DHT11	15
main/drivers/dht11.h	
Interfaz pública del controlador para el sensor DHT11	18
main/drivers/ds3231.c	
Implementación del driver RTC DS3231	20
main/drivers/ds3231.h	
Interfaz pública del controlador para el RTC DS3231	27
main/drivers/lcd_parallel.c	
Implementación del driver para pantalla LCD 16x2 basada en HD44780	33
main/drivers/lcd_parallel.h	
Driver para pantalla LCD 16x2 en modo paralelo (4 bits)	39
main/drivers/sd_card.c	
Implementación del módulo de almacenamiento en tarjeta SD	42
main/drivers/sd_card.h	
Módulo de gestión de tarjeta SD mediante interfaz SPI	50
main/include/common.h	
Definiciones y estructuras compartidas del sistema	56
main/include/queues.h	
Declaración de las colas FreeRTOS utilizadas por el sistema	57
main/mqtt/mqtt_manager.c	
Implementación del gestor de comunicación MQTT	66
main/mqtt/mqtt_manager.h	
Gestión de la comunicación MQTT del sistema	71
main/tasks/task_display.c	
Tarea encargada de la visualización de datos en LCD	74
main/tasks/task_logger.c	
Tarea encargada del almacenamiento de datos en tarjeta SD	76
main/tasks/task_mqtt.c	
Tarea encargada de la transmisión MQTT de datos ambientales	77
main/tasks/task_sensores.c	
Tarea de adquisición de datos ambientales	81
main/wifi/wifi_manager.c	
Implementación del gestor de conectividad WiFi	83
main/wifi/wifi_manager.h	
Gestión de conectividad WiFi para el ESP32	89

Capítulo 4

Documentación de directorios

4.1. Referencia del directorio main/drivers

Archivos

- archivo [dht11.c](#)
Implementación del driver para sensor DHT11.
- archivo [dht11.h](#)
Interfaz pública del controlador para el sensor DHT11.
- archivo [ds3231.c](#)
Implementación del driver RTC DS3231.
- archivo [ds3231.h](#)
Interfaz pública del controlador para el RTC DS3231.
- archivo [lcd_parallel.c](#)
Implementación del driver para pantalla LCD 16x2 basada en HD44780.
- archivo [lcd_parallel.h](#)
Driver para pantalla LCD 16x2 en modo paralelo (4 bits).
- archivo [sd_card.c](#)
Implementación del módulo de almacenamiento en tarjeta SD.
- archivo [sd_card.h](#)
Módulo de gestión de tarjeta SD mediante interfaz SPI.

4.2. Referencia del directorio main/include

Archivos

- archivo [common.h](#)
Definiciones y estructuras compartidas del sistema.
- archivo [queues.h](#)
Declaración de las colas FreeRTOS utilizadas por el sistema.

4.3. Referencia del directorio main

Directorios

- directorio [drivers](#)
- directorio [include](#)
- directorio [mqtt](#)
- directorio [tasks](#)
- directorio [wifi](#)

Archivos

- archivo [main.c](#)
Punto de entrada del sistema ESP32 para monitoreo ambiental.

4.4. Referencia del directorio main/mqtt

Archivos

- archivo [mqtt_manager.c](#)
Implementación del gestor de comunicación MQTT.
- archivo [mqtt_manager.h](#)
Gestión de la comunicación MQTT del sistema.

4.5. Referencia del directorio main/tasks

Archivos

- archivo [task_display.c](#)
Tarea encargada de la visualización de datos en LCD.
- archivo [task_logger.c](#)
Tarea encargada del almacenamiento de datos en tarjeta SD.
- archivo [task_mqtt.c](#)
Tarea encargada de la transmisión MQTT de datos ambientales.
- archivo [task_sensores.c](#)
Tarea de adquisición de datos ambientales.

4.6. Referencia del directorio main/wifi

Archivos

- archivo [wifi_manager.c](#)
Implementación del gestor de conectividad WiFi.
- archivo [wifi_manager.h](#)
Gestión de conectividad WiFi para el ESP32.

Capítulo 5

Documentación de clases

5.1. Referencia de la estructura datos_ambiente_t

Estructura que almacena una medición ambiental completa.

```
#include <common.h>
```

Atributos públicos

- float [temperatura](#)
Temperatura ambiente.
- float [humedad](#)
Humedad relativa.
- uint8_t [hora](#)
Hora de la medición.
- uint8_t [minuto](#)
Minuto de la medición.
- uint8_t [segundo](#)
Segundo de la medición.
- uint8_t [dia](#)
Día del mes.
- uint8_t [mes](#)
Mes de la medición.
- uint16_t [anio](#)
Año de la medición.

5.1.1. Descripción detallada

Estructura que almacena una medición ambiental completa.

Contiene los datos adquiridos por los sensores junto con la marca temporal asociada obtenida desde el RTC DS3231.

Esta estructura es utilizada para:

- Transferencia de datos entre tareas FreeRTOS.
- Almacenamiento en tarjeta SD.
- Publicación mediante MQTT.
- Visualización en la interfaz de usuario.

5.1.2. Documentación de datos miembro

5.1.2.1. anio

```
uint16_t datos_ambiente_t::anio
```

Año de la medición.

5.1.2.2. dia

```
uint8_t datos_ambiente_t::dia
```

Día del mes.

5.1.2.3. hora

```
uint8_t datos_ambiente_t::hora
```

Hora de la medición.

Formato de 24 horas.

5.1.2.4. humedad

```
float datos_ambiente_t::humedad
```

Humedad relativa.

Unidad: porcentaje (RH).

5.1.2.5. mes

```
uint8_t datos_ambiente_t::mes
```

Mes de la medición.

5.1.2.6. minuto

```
uint8_t datos_ambiente_t::minuto
```

Minuto de la medición.

5.1.2.7. segundo

```
uint8_t datos_ambiente_t::segundo
```

Segundo de la medición.

5.1.2.8. temperatura

```
float datos_ambiente_t::temperatura
```

Temperatura ambiente.

Unidad: grados Celsius (°C).

La documentación de esta estructura está generada del siguiente archivo:

- main/include/[common.h](#)

5.2. Referencia de la estructura dht11_data_t

Estructura que almacena las mediciones obtenidas del sensor DHT11.

```
#include <dht11.h>
```

Atributos públicos

- float [temperatura](#)
Temperatura ambiente medida.
- float [humedad](#)
Humedad relativa medida.

5.2.1. Descripción detallada

Estructura que almacena las mediciones obtenidas del sensor DHT11.

Esta estructura contiene los valores ambientales reportados por el sensor después de una lectura válida.

5.2.2. Documentación de datos miembro

5.2.2.1. humedad

```
float dht11_data_t::humedad
```

Humedad relativa medida.

Unidad: porcentaje (RH).

5.2.2.2. temperatura

```
float dht11_data_t::temperatura
```

Temperatura ambiente medida.

Unidad: grados Celsius (°C).

La documentación de esta estructura está generada del siguiente archivo:

- main/drivers/[dht11.h](#)

5.3. Referencia de la estructura rtc_time_t

Representa una fecha y hora completas.

```
#include <ds3231.h>
```

Atributos públicos

- uint8_t segundos
Segundos.
- uint8_t minutos
Minutos.
- uint8_t horas
Horas.
- uint8_t dia
Día del mes.
- uint8_t mes
Mes del año.
- uint16_t anio
Año completo.

5.3.1. Descripción detallada

Representa una fecha y hora completas.

Esta estructura almacena los valores obtenidos desde los registros internos del RTC DS3231 o los valores que serán escritos en él.

5.3.2. Documentación de datos miembro

5.3.2.1. anio

```
uint16_t rtc_time_t::anio
```

Año completo.

Ejemplo: 2026.

5.3.2.2. dia

```
uint8_t rtc_time_t::dia
```

Día del mes.

Rango válido: 1 - 31.

5.3.2.3. horas

```
uint8_t rtc_time_t::horas
```

Horas.

Formato de 24 horas. Rango válido: 0 - 23.

5.3.2.4. mes

```
uint8_t rtc_time_t::mes
```

Mes del año.

Rango válido: 1 - 12.

5.3.2.5. minutos

```
uint8_t rtc_time_t::minutos
```

Minutos.

Rango válido: 0 - 59.

5.3.2.6. segundos

```
uint8_t rtc_time_t::segundos
```

Segundos.

Rango válido: 0 - 59.

La documentación de esta estructura está generada del siguiente archivo:

- [main/drivers/ds3231.h](#)

Capítulo 6

Documentación de archivos

6.1. Referencia del archivo main/drivers/dht11.c

Implementación del driver para sensor DHT11.

```
#include "dht11.h"  
#include "driver/gpio.h"  
#include "esp_rom_sys.h"
```

defines

- #define `DHT11_GPIO` `GPIO_NUM_5`
GPIO utilizado para la comunicación con el sensor DHT11.

Funciones

- static void `set_output` ()
Configura el pin de datos como salida.
- static void `set_input` ()
Configura el pin de datos como entrada.
- int `dht11_read` (`dht11_data_t` *data)
Realiza una lectura completa del sensor DHT11.

6.1.1. Descripción detallada

Implementación del driver para sensor DHT11.

Este módulo implementa la comunicación de bajo nivel con el sensor DHT11 utilizando el protocolo propietario basado en temporización de pulsos.

El controlador realiza:

- Generación de la señal de inicio.
- Recepción de los 40 bits transmitidos por el sensor.
- Verificación de integridad mediante checksum.
- Conversión de los datos recibidos a temperatura y humedad.

Nota

El protocolo del DHT11 depende de retardos en microsegundos.
Se recomienda evitar interrupciones prolongadas durante la lectura.

Atención

El acceso concurrente al sensor debe protegerse mediante mutex cuando sea utilizado desde múltiples tareas FreeRTOS.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.1.2. Documentación de «define»

6.1.2.1. DHT11_GPIO

```
#define DHT11_GPIO GPIO_NUM_5
```

GPIO utilizado para la comunicación con el sensor DHT11.

6.1.3. Documentación de funciones

6.1.3.1. dht11_read()

```
int dht11_read (  
    dht11_data_t * data)
```

Realiza una lectura completa del sensor DHT11.

Ejecuta la secuencia completa de comunicación:

1. Envía la señal de inicio al sensor.
2. Espera la respuesta de presencia.
3. Lee los 40 bits de información transmitidos.
4. Verifica la integridad mediante checksum.
5. Extrae los valores de temperatura y humedad.

Los datos obtenidos se almacenan en la estructura proporcionada por el usuario.

Parámetros

out	data	Estructura donde se almacenarán los valores leídos.
-----	------	---

Valores devueltos

0	Lectura realizada correctamente.
-1	No se recibió respuesta del sensor.
-2	Error de checksum en los datos recibidos.

Nota

Esta función es bloqueante debido al uso de retardos de microsegundos requeridos por el protocolo DHT11.

Esta función ejecuta el protocolo de comunicación requerido por el sensor, recibe los datos transmitidos y verifica su integridad mediante checksum.

Si la lectura es exitosa, los valores obtenidos son almacenados en la estructura indicada por el parámetro `data`.

Parámetros

out	data	Puntero a la estructura donde se almacenarán los datos leídos de temperatura y humedad.
-----	------	---

Valores devueltos

0	Lectura realizada correctamente.
-1	Error de comunicación o ausencia de respuesta del sensor.
-2	Error de verificación de checksum.

Atención

Esta función es bloqueante debido a los requerimientos de temporización del protocolo DHT11.

Nota

Se recomienda proteger el acceso mediante mutex si el sensor puede ser utilizado desde múltiples tareas FreeRTOS.

6.1.3.2. set_input()

```
void set_input () [static]
```

Configura el pin de datos como entrada.

Esta función se utiliza después de la señal de inicio para permitir que el sensor transmita los datos hacia el ESP32.

6.1.3.3. `set_output()`

```
void set_output () [static]
```

Configura el pin de datos como salida.

Esta función se utiliza durante la fase de inicio de comunicación, cuando el ESP32 debe enviar la señal de arranque al sensor.

6.2. Referencia del archivo `main/drivers/dht11.h`

Interfaz pública del controlador para el sensor DHT11.

```
#include <stdint.h>
```

Clases

- `struct dht11_data_t`

Estructura que almacena las mediciones obtenidas del sensor DHT11.

Funciones

- `int dht11_read (dht11_data_t *data)`

Realiza una lectura completa del sensor DHT11.

6.2.1. Descripción detallada

Interfaz pública del controlador para el sensor DHT11.

Este módulo proporciona las definiciones y funciones necesarias para obtener mediciones de temperatura y humedad relativa utilizando el sensor DHT11.

El sensor utiliza un protocolo propietario de comunicación basado en temporización precisa de pulsos digitales.

Nota

La implementación del protocolo se encuentra en [dht11.c](#).

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.2.2. Documentación de funciones

6.2.2.1. dht11_read()

```
int dht11_read (  
    dht11_data_t * data)
```

Realiza una lectura completa del sensor DHT11.

Esta función ejecuta el protocolo de comunicación requerido por el sensor, recibe los datos transmitidos y verifica su integridad mediante checksum.

Si la lectura es exitosa, los valores obtenidos son almacenados en la estructura indicada por el parámetro `data`.

Parámetros

out	data	Puntero a la estructura donde se almacenarán los datos leídos de temperatura y humedad.
-----	------	---

Valores devueltos

0	Lectura realizada correctamente.
-1	Error de comunicación o ausencia de respuesta del sensor.
-2	Error de verificación de checksum.

Atención

Esta función es bloqueante debido a los requerimientos de temporización del protocolo DHT11.

Nota

Se recomienda proteger el acceso mediante mutex si el sensor puede ser utilizado desde múltiples tareas FreeRTOS.

Ejecuta la secuencia completa de comunicación:

1. Envía la señal de inicio al sensor.
2. Espera la respuesta de presencia.
3. Lee los 40 bits de información transmitidos.
4. Verifica la integridad mediante checksum.
5. Extrae los valores de temperatura y humedad.

Los datos obtenidos se almacenan en la estructura proporcionada por el usuario.

Parámetros

out	data	Estructura donde se almacenarán los valores leídos.
-----	------	---

Valores devueltos

0	Lectura realizada correctamente.
-1	No se recibió respuesta del sensor.
-2	Error de checksum en los datos recibidos.

Nota

Esta función es bloqueante debido al uso de retardos de microsegundos requeridos por el protocolo DHT11.

6.3. dht11.h

[Ir a la documentación de este archivo.](#)

```
00001
00019
00020 #pragma once
00021
00022 #include <stdint.h>
00023
00030 typedef struct
00031 {
00037     float temperatura;
00038
00044     float humedad;
00045
00046 } dht11_data_t;
00047
00071 int dht11_read(dht11_data_t *data);
```

6.4. Referencia del archivo main/drivers/ds3231.c

Implementación del driver RTC DS3231.

```
#include "ds3231.h"
#include "driver/i2c.h"
#include "esp_log.h"
```

defines

- **#define TAG** "DS3231"
Etiqueta utilizada para mensajes de depuración.
- **#define DS3231_ADDR** 0x68
Dirección I2C del RTC DS3231.
- **#define I2C_PORT** I2C_NUM_0
Puerto I2C utilizado para la comunicación.

Funciones

- static uint8_t `bcd_to_dec` (uint8_t val)
Convierte un valor codificado en BCD a decimal.
- static uint8_t `dec_to_bcd` (uint8_t val)
Convierte un valor decimal a formato BCD.
- int `ds3231_read_time` (rtc_time_t *time)
Obtiene la fecha y hora actuales del DS3231.
- int `ds3231_set_time` (rtc_time_t *time)
Configura la fecha y hora del RTC DS3231.
- int `ds3231_set_alarm_every_minute` ()
Configura una alarma periódica cada minuto.
- int `ds3231_clear_alarm_flag` ()
Limpia el indicador de alarma A1F.

6.4.1. Descripción detallada

Implementación del driver RTC DS3231.

Maneja comunicación I2C para:

- Lectura de tiempo
- Configuración de alarmas

Nota

Debe protegerse con mutex si se usa en múltiples tareas.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.4.2. Documentación de «define»

6.4.2.1. DS3231_ADDR

```
#define DS3231_ADDR 0x68
```

Dirección I2C del RTC DS3231.

6.4.2.2. I2C_PORT

```
#define I2C_PORT I2C_NUM_0
```

Puerto I2C utilizado para la comunicación.

6.4.2.3. TAG

```
#define TAG "DS3231"
```

Etiqueta utilizada para mensajes de depuración.

6.4.3. Arquitectura del sistema

El módulo DS3231 constituye la referencia temporal principal del sistema distribuido de fermentación de café.

Sus funciones principales son:

- Generación de timestamps para almacenamiento en SD.
- Sincronización de mediciones ambientales.
- Activación periódica del sistema mediante alarmas.
- Soporte para estrategias de ahorro energético basadas en sleep.

6.4.4. Documentación de funciones

6.4.4.1. bcd_to_dec()

```
uint8_t bcd_to_dec (  
    uint8_t val) [static]
```

Convierte un valor codificado en BCD a decimal.

El DS3231 almacena los valores de fecha y hora en formato Binary Coded Decimal (BCD), por lo que es necesario convertirlos antes de utilizarlos en la aplicación.

Parámetros

<i>val</i>	Valor en formato BCD.
------------	-----------------------

Devuelve

Valor convertido a decimal.

6.4.4.2. dec_to_bcd()

```
uint8_t dec_to_bcd (
    uint8_t val) [static]
```

Convierte un valor decimal a formato BCD.

Esta función se utiliza antes de escribir valores de fecha y hora en los registros internos del DS3231.

Parámetros

<i>val</i>	Valor decimal.
------------	----------------

Devuelve

Valor convertido a BCD.

6.4.4.3. ds3231_clear_alarm_flag()

```
int ds3231_clear_alarm_flag (
    void )
```

Limpia el indicador de alarma A1F.

Limpia el indicador interno de alarma.

Después de que una alarma genera una interrupción, el bit A1F permanece activado dentro del registro de estado del DS3231.

Esta función limpia dicho bit para permitir que la siguiente alarma vuelva a producir una interrupción válida.

Registro utilizado:

- Status Register (0x0F)

Valores devueltos

0	Operación completada.
---	-----------------------

Atención

Si esta función no se ejecuta después de atender la interrupción, las alarmas posteriores no funcionarán correctamente.

Limpia el indicador de alarma A1F.

Después de generarse una interrupción, el DS3231 mantiene activo el bit A1F dentro del registro de estado.

Esta función limpia dicho bit para permitir que futuras alarmas vuelvan a generar interrupciones.

Valores devueltos

0	Operación completada.
---	-----------------------

Atención

Si no se ejecuta esta función después de atender una alarma, las siguientes interrupciones pueden no producirse.

6.4.4.4. ds3231_read_time()

```
int ds3231_read_time (  
    rtc_time_t * time)
```

Obtiene la fecha y hora actuales del DS3231.

Lee la fecha y hora actuales almacenadas en el RTC.

Lee los registros de tiempo del RTC mediante comunicación I2C y convierte los valores almacenados en formato BCD a decimal.

Los campos recuperados son:

- Segundos
- Minutos
- Horas
- Día del mes
- Mes
- Año

Parámetros

out	time	Estructura donde se almacenará la fecha y hora.
-----	------	---

Valores devueltos

0	Lectura exitosa.
-1	Error de comunicación I2C.

Nota

Esta función debe ejecutarse bajo protección de mutex si el bus I2C es compartido con otros dispositivos.

Obtiene la fecha y hora actuales del DS3231.

Recupera los registros de tiempo del DS3231 mediante I2C, convirtiendo automáticamente los datos desde formato BCD a formato decimal.

Parámetros

out	time	Estructura donde se almacenará la fecha y hora.
-----	------	---

Valores devueltos

0	Lectura exitosa.
-1	Error de comunicación I2C.

6.4.4.5. ds3231_set_alarm_every_minute()

```
int ds3231_set_alarm_every_minute (
    void )
```

Configura una alarma periódica cada minuto.

Programa la alarma 1 del DS3231 para generar una interrupción cuando los segundos alcancen el valor 0.

La configuración utilizada permite generar una interrupción exactamente al inicio de cada minuto.

Registros utilizados:

- Alarm1 (0x07 - 0x0A)
- Control (0x0E)

Esta funcionalidad se utiliza para despertar periódicamente al ESP32 desde modos de bajo consumo.

Valores devueltos

0	Configuración exitosa.
-1	Error de comunicación I2C.

Nota

El pin INT/SQW del DS3231 debe estar conectado a un GPIO configurado como interrupción externa.

Utiliza la alarma 1 del DS3231 para generar una interrupción exactamente cuando los segundos alcanzan el valor cero.

Esta funcionalidad es utilizada para despertar el sistema desde modos de bajo consumo y ejecutar los ciclos de medición.

Valores devueltos

0	Configuración exitosa.
-1	Error de comunicación I2C.

Nota

Requiere que el pin INT/SQW del DS3231 esté conectado a un GPIO configurado como interrupción externa.

6.4.4.6. ds3231_set_time()

```
int ds3231_set_time (
    rtc_time_t * time)
```

Configura la fecha y hora del RTC DS3231.

Configura la fecha y hora del RTC.

Convierte los valores de la estructura `rtc_time_t` a formato BCD y los almacena en los registros internos del RTC.

Parámetros

in	<i>time</i>	Estructura que contiene la fecha y hora a configurar.
----	-------------	---

Valores devueltos

0	Configuración exitosa.
-1	Error de comunicación I2C.

Atención

Esta operación modifica la referencia temporal utilizada por todo el sistema para generación de timestamps y alarmas.

Configura la fecha y hora del RTC DS3231.

Convierte los valores contenidos en la estructura `rtc_time_t` al formato requerido por el DS3231 y los almacena en sus registros internos.

Parámetros

<code>in</code>	<code>time</code>	Fecha y hora a configurar.
-----------------	-------------------	----------------------------

Valores devueltos

<code>0</code>	Configuración exitosa.
<code>-1</code>	Error de comunicación I2C.

Atención

Modificar la hora afecta la generación de timestamps, alarmas e historial de datos almacenados por el sistema.

6.5. Referencia del archivo main/drivers/ds3231.h

Interfaz pública del controlador para el RTC DS3231.

```
#include <stdint.h>
```

Clases

- struct `rtc_time_t`
Representa una fecha y hora completas.

Funciones

- int `ds3231_read_time` (`rtc_time_t` *time)
Lee la fecha y hora actuales almacenadas en el RTC.
- int `ds3231_set_time` (`rtc_time_t` *time)
Configura la fecha y hora del RTC.
- int `ds3231_set_alarm_every_minute` (void)
Configura una alarma periódica cada minuto.
- int `ds3231_clear_alarm_flag` (void)
Limpia el indicador interno de alarma.

6.5.1. Descripción detallada

Interfaz pública del controlador para el RTC DS3231.

Este módulo proporciona las funciones necesarias para interactuar con el reloj de tiempo real (RTC) DS3231 mediante comunicación I2C.

Funcionalidades principales:

- Lectura de fecha y hora.
- Configuración de fecha y hora.
- Configuración de alarmas periódicas.
- Gestión de interrupciones generadas por alarmas.

Nota

El acceso al bus I2C debe protegerse mediante mutex cuando sea compartido entre múltiples tareas Free↔RTOS.

6.5.2. Arquitectura del sistema

En el sistema de fermentación de café, el DS3231 actúa como referencia temporal principal para:

- Generación de timestamps para almacenamiento en SD.
- Sincronización de mediciones de sensores.
- Activación periódica mediante interrupciones.
- Implementación de estrategias de ahorro energético.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.5.3. Documentación de funciones

6.5.3.1. ds3231_clear_alarm_flag()

```
int ds3231_clear_alarm_flag (  
    void )
```

Limpia el indicador interno de alarma.

Limpia el indicador de alarma A1F.

Después de generarse una interrupción, el DS3231 mantiene activo el bit A1F dentro del registro de estado.

Esta función limpia dicho bit para permitir que futuras alarmas vuelvan a generar interrupciones.

Valores devueltos

0	Operación completada.
---	-----------------------

Atención

Si no se ejecuta esta función después de atender una alarma, las siguientes interrupciones pueden no producirse.

Limpia el indicador interno de alarma.

Después de que una alarma genera una interrupción, el bit A1F permanece activado dentro del registro de estado del DS3231.

Esta función limpia dicho bit para permitir que la siguiente alarma vuelva a producir una interrupción válida.

Registro utilizado:

- Status Register (0x0F)

Valores devueltos

0	Operación completada.
---	-----------------------

Atención

Si esta función no se ejecuta después de atender la interrupción, las alarmas posteriores no funcionarán correctamente.

6.5.3.2. ds3231_read_time()

```
int ds3231_read_time (
    rtc_time_t * time)
```

Lee la fecha y hora actuales almacenadas en el RTC.

Obtiene la fecha y hora actuales del DS3231.

Recupera los registros de tiempo del DS3231 mediante I2C, convirtiendo automáticamente los datos desde formato BCD a formato decimal.

Parámetros

out	time	Estructura donde se almacenará la fecha y hora.
-----	------	---

Valores devueltos

0	Lectura exitosa.
-1	Error de comunicación I2C.

Lee la fecha y hora actuales almacenadas en el RTC.

Lee los registros de tiempo del RTC mediante comunicación I2C y convierte los valores almacenados en formato BCD a decimal.

Los campos recuperados son:

- Segundos
- Minutos
- Horas
- Día del mes
- Mes
- Año

Parámetros

out	time	Estructura donde se almacenará la fecha y hora.
-----	------	---

Valores devueltos

0	Lectura exitosa.
-1	Error de comunicación I2C.

Nota

Esta función debe ejecutarse bajo protección de mutex si el bus I2C es compartido con otros dispositivos.

6.5.3.3. ds3231_set_alarm_every_minute()

```
int ds3231_set_alarm_every_minute (  
    void )
```

Configura una alarma periódica cada minuto.

Utiliza la alarma 1 del DS3231 para generar una interrupción exactamente cuando los segundos alcanzan el valor cero.

Esta funcionalidad es utilizada para despertar el sistema desde modos de bajo consumo y ejecutar los ciclos de medición.

Valores devueltos

0	Configuración exitosa.
-1	Error de comunicación I2C.

Nota

Requiere que el pin INT/SQW del DS3231 esté conectado a un GPIO configurado como interrupción externa.

Programa la alarma 1 del DS3231 para generar una interrupción cuando los segundos alcancen el valor 0.

La configuración utilizada permite generar una interrupción exactamente al inicio de cada minuto.

Registros utilizados:

- Alarm1 (0x07 - 0x0A)
- Control (0x0E)

Esta funcionalidad se utiliza para despertar periódicamente al ESP32 desde modos de bajo consumo.

Valores devueltos

0	Configuración exitosa.
-1	Error de comunicación I2C.

Nota

El pin INT/SQW del DS3231 debe estar conectado a un GPIO configurado como interrupción externa.

6.5.3.4. ds3231_set_time()

```
int ds3231_set_time (  
    rtc_time_t * time)
```

Configura la fecha y hora del RTC.

Configura la fecha y hora del RTC DS3231.

Convierte los valores contenidos en la estructura `rtc_time_t` al formato requerido por el DS3231 y los almacena en sus registros internos.

Parámetros

in	time	Fecha y hora a configurar.
----	------	----------------------------

Valores devueltos

0	Configuración exitosa.
-1	Error de comunicación I2C.

Atención

Modificar la hora afecta la generación de timestamps, alarmas e historial de datos almacenados por el sistema.

Configura la fecha y hora del RTC.

Convierte los valores de la estructura `rtc_time_t` a formato BCD y los almacena en los registros internos del RTC.

Parámetros

in	<i>time</i>	Estructura que contiene la fecha y hora a configurar.
----	-------------	---

Valores devueltos

0	Configuración exitosa.
-1	Error de comunicación I2C.

Atención

Esta operación modifica la referencia temporal utilizada por todo el sistema para generación de timestamps y alarmas.

6.6. ds3231.h

[Ir a la documentación de este archivo.](#)

```

00001
00032
00033 #ifndef DS3231_H
00034 #define DS3231_H
00035
00036 #include <stdint.h>
00037
00044 typedef struct
00045 {
00051     uint8_t segundos;
00052
00058     uint8_t minutos;
00059
00066     uint8_t horas;
00067
00073     uint8_t dia;
00074
00080     uint8_t mes;
00081
00087     uint16_t anio;
00088
00089 } rtc_time_t;
00090
00103 int ds3231_read_time(rtc_time_t *time);
00104
00120 int ds3231_set_time(rtc_time_t *time);
00121
00137 int ds3231_set_alarm_every_minute(void);
00138
00153 int ds3231_clear_alarm_flag(void);
00154
00155 #endif

```


6.7. Referencia del archivo main/drivers/lcd_parallel.c

Implementación del driver para pantalla LCD 16x2 basada en HD44780.

```
#include "lcd_parallel.h"
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_rom_sys.h"
```

defines

- #define LCD_RS GPIO_NUM_13
- #define LCD_E GPIO_NUM_33
- #define LCD_D4 GPIO_NUM_14
- #define LCD_D5 GPIO_NUM_32
- #define LCD_D6 GPIO_NUM_26
- #define LCD_D7 GPIO_NUM_25

Funciones

- static void `lcd_send_nibble` (uint8_t nibble)
Envía un nibble (4 bits) al LCD.
- static void `lcd_send_byte` (uint8_t data, int rs)
Envía un byte completo al controlador LCD.
- static void `lcd_cmd` (uint8_t cmd)
Envía un comando al controlador HD44780.
- static void `lcd_data` (uint8_t data)
Envía un carácter al display.
- void `lcd_init` (void)
Inicializa la pantalla LCD.
- void `lcd_clear` (void)
Limpia completamente el contenido de la pantalla.
- void `lcd_set_cursor` (uint8_t col, uint8_t row)
Posiciona el cursor en una ubicación específica.
- void `lcd_print` (const char *str)
Imprime una cadena de texto en la pantalla.

6.7.1. Descripción detallada

Implementación del driver para pantalla LCD 16x2 basada en HD44780.

Este módulo implementa la comunicación paralela en modo de 4 bits utilizando GPIO del ESP32 para controlar una pantalla LCD 16x2.

Funcionalidades implementadas:

- Inicialización del controlador HD44780.
- Envío de comandos.
- Envío de caracteres.
- Posicionamiento del cursor.
- Limpieza de pantalla.

6.7.2. Rol dentro del sistema

Este módulo forma parte del ESP32 externo y proporciona la interfaz visual local para el monitoreo del proceso de fermentación.

La información mostrada puede incluir:

- Temperatura ambiente.
- Humedad relativa.
- Estado de conexión WiFi.
- Estado MQTT.
- Estado del almacenamiento SD.
- Mensajes de diagnóstico.

Nota

No es seguro para acceso concurrente.

Se recomienda que únicamente la tarea `task_display` acceda directamente a este módulo.

Atención

La comunicación requiere retardos precisos para cumplir las especificaciones del controlador HD44780.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.7.3. Documentación de «define»

6.7.3.1. LCD_D4

```
#define LCD_D4 GPIO_NUM_14
```

Data bit 4

6.7.3.2. LCD_D5

```
#define LCD_D5 GPIO_NUM_32
```

Data bit 5

6.7.3.3. LCD_D6

```
#define LCD_D6 GPIO_NUM_26
```

Data bit 6

6.7.3.4. LCD_D7

```
#define LCD_D7 GPIO_NUM_25
```

Data bit 7

6.7.3.5. LCD_E

```
#define LCD_E GPIO_NUM_33
```

Pin Enable

6.7.3.6. LCD_RS

```
#define LCD_RS GPIO_NUM_13
```

Pin Register Select

6.7.4. Documentación de funciones

6.7.4.1. lcd_clear()

```
void lcd_clear (  
    void )
```

Limpia completamente el contenido de la pantalla.

Borra todos los caracteres visibles y reposiciona el cursor en la primera columna de la primera fila.

Borra todos los caracteres visibles y reposiciona el cursor en la posición inicial de la primera fila.

Nota

Esta operación requiere varios milisegundos debido a las limitaciones temporales del controlador HD44780.

6.7.4.2. `lcd_cmd()`

```
void lcd_cmd (
    uint8_t cmd) [static]
```

Envía un comando al controlador HD44780.

Permite ejecutar operaciones internas como:

- Limpiar pantalla.
- Posicionar cursor.
- Configurar modos de operación.

Parámetros

<i>cmd</i>	Código de comando HD44780.
------------	----------------------------

6.7.4.3. `lcd_data()`

```
void lcd_data (
    uint8_t data) [static]
```

Envía un carácter al display.

El dato es almacenado en la memoria DDRAM interna del controlador y mostrado en la posición actual del cursor.

Parámetros

<i>data</i>	Código ASCII del carácter a visualizar.
-------------	---

6.7.4.4. `lcd_init()`

```
void lcd_init (
    void )
```

Inicializa la pantalla LCD.

Configura los GPIO asociados a las líneas de control y datos, ejecuta la secuencia de arranque recomendada por el fabricante y habilita el funcionamiento en modo de 4 bits.

La secuencia implementada sigue las recomendaciones del controlador HD44780.

Nota

Debe ejecutarse una única vez durante el arranque del sistema.

Configura los GPIO asociados a las líneas de datos y control, ejecutando posteriormente la secuencia de inicialización definida por el controlador HD44780 para operación en modo de 4 bits.

Esta función debe ejecutarse antes de utilizar cualquier otra función del módulo.

Nota

Debe llamarse una única vez durante la fase de arranque del sistema.

6.7.4.5. lcd_print()

```
void lcd_print (
    const char * str)
```

Imprime una cadena de texto en la pantalla.

Imprime una cadena de caracteres en la pantalla.

Los caracteres son enviados secuencialmente comenzando desde la posición actual del cursor.

Parámetros

<i>str</i>	Cadena terminada en carácter nulo ('\0').
------------	---

Atención

No se realiza control automático de longitud ni salto de línea.

Imprime una cadena de texto en la pantalla.

Los caracteres son enviados secuencialmente al controlador LCD comenzando desde la posición actual del cursor.

Parámetros

<i>str</i>	Cadena de caracteres terminada en '\0'.
------------	---

Nota

No se realiza control automático de longitud ni salto de línea. Es responsabilidad de la aplicación garantizar que el texto se ajuste al tamaño disponible del display.

6.7.4.6. lcd_send_byte()

```
void lcd_send_byte (
    uint8_t data,
    int rs) [static]
```

Envía un byte completo al controlador LCD.

Debido al uso del modo de comunicación de 4 bits, el byte es dividido en dos nibbles que son transmitidos secuencialmente.

Parámetros

<i>data</i>	Byte a transmitir.
-------------	--------------------

<i>rs</i>	Selección de registro: • 0: Comando. • 1: Dato.
-----------	--

6.7.4.7. `lcd_send_nibble()`

```
void lcd_send_nibble (
    uint8_t nibble) [static]
```

Envía un nibble (4 bits) al LCD.

Esta función coloca los bits en las líneas D4-D7 y genera un pulso en el pin Enable para registrar el dato.

Parámetros

<i>nibble</i>	Valor de 4 bits a enviar (LSB alineado)
---------------	---

Nota

Función de bajo nivel crítica para la comunicación

6.7.4.8. `lcd_set_cursor()`

```
void lcd_set_cursor (
    uint8_t col,
    uint8_t row)
```

Posiciona el cursor en una ubicación específica.

Convierte la fila y columna solicitadas a la dirección DDRAM correspondiente del controlador HD44780.

Parámetros

<i>col</i>	Columna destino (0-15).
<i>row</i>	Fila destino (0-1).

Convierte automáticamente la fila y columna indicadas a la dirección DDRAM correspondiente del controlador LCD.

Parámetros

<i>col</i>	Columna destino. Rango válido: 0–15.
<i>row</i>	Fila destino. Rango válido: 0–1.

Nota

Valores fuera de rango pueden producir comportamientos inesperados dependiendo de la implementación.

6.8. Referencia del archivo main/drivers/lcd_parallel.h

Driver para pantalla LCD 16x2 en modo paralelo (4 bits).

```
#include <stdint.h>
```

Funciones

- void `lcd_init` (void)
Inicializa la pantalla LCD.
- void `lcd_clear` (void)
Limpia completamente el contenido de la pantalla.
- void `lcd_set_cursor` (uint8_t col, uint8_t row)
Posiciona el cursor en una ubicación específica.
- void `lcd_print` (const char *str)
Imprime una cadena de caracteres en la pantalla.

6.8.1. Descripción detallada

Driver para pantalla LCD 16x2 en modo paralelo (4 bits).

Este módulo proporciona una interfaz para controlar una pantalla LCD basada en el controlador HD44780 utilizando comunicación paralela de 4 bits.

Funcionalidades principales:

- Inicialización del display.
- Posicionamiento del cursor.
- Impresión de texto.
- Limpieza de pantalla.

6.8.2. Integración en el sistema

Dentro del sistema distribuido de fermentación de café, esta pantalla forma parte del ESP32 externo y permite visualizar información de:

- Temperatura ambiente.
- Humedad relativa.
- Estado de conexión WiFi.
- Estado de comunicación MQTT.
- Estado de almacenamiento en SD.
- Información de diagnóstico.

Nota

Este módulo no es thread-safe. Si es utilizado desde múltiples tareas FreeRTOS debe protegerse mediante un mutex.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.8.3. Documentación de funciones

6.8.3.1. lcd_clear()

```
void lcd_clear (
    void )
```

Limpia completamente el contenido de la pantalla.

Borra todos los caracteres visibles y reposiciona el cursor en la posición inicial de la primera fila.

Nota

Esta operación requiere varios milisegundos debido a las limitaciones temporales del controlador HD44780.

Borra todos los caracteres visibles y reposiciona el cursor en la primera columna de la primera fila.

6.8.3.2. lcd_init()

```
void lcd_init (
    void )
```

Inicializa la pantalla LCD.

Configura los GPIO asociados a las líneas de datos y control, ejecutando posteriormente la secuencia de inicialización definida por el controlador HD44780 para operación en modo de 4 bits.

Esta función debe ejecutarse antes de utilizar cualquier otra función del módulo.

Nota

Debe llamarse una única vez durante la fase de arranque del sistema.

Configura los GPIO asociados a las líneas de control y datos, ejecuta la secuencia de arranque recomendada por el fabricante y habilita el funcionamiento en modo de 4 bits.

La secuencia implementada sigue las recomendaciones del controlador HD44780.

Nota

Debe ejecutarse una única vez durante el arranque del sistema.

6.8.3.3. lcd_print()

```
void lcd_print (
    const char * str)
```

Imprime una cadena de caracteres en la pantalla.

Imprime una cadena de texto en la pantalla.

Los caracteres son enviados secuencialmente al controlador LCD comenzando desde la posición actual del cursor.

Parámetros

<i>str</i>	Cadena de caracteres terminada en '\0'.
------------	---

Nota

No se realiza control automático de longitud ni salto de línea. Es responsabilidad de la aplicación garantizar que el texto se ajuste al tamaño disponible del display.

Imprime una cadena de caracteres en la pantalla.

Los caracteres son enviados secuencialmente comenzando desde la posición actual del cursor.

Parámetros

<i>str</i>	Cadena terminada en carácter nulo ('\0').
------------	---

Atención

No se realiza control automático de longitud ni salto de línea.

6.8.3.4. lcd_set_cursor()

```
void lcd_set_cursor (
    uint8_t col,
    uint8_t row)
```

Posiciona el cursor en una ubicación específica.

Convierte automáticamente la fila y columna indicadas a la dirección DDRAM correspondiente del controlador LCD.

Parámetros

<i>col</i>	Columna destino. Rango válido: 0–15.
<i>row</i>	Fila destino. Rango válido: 0–1.

Nota

Valores fuera de rango pueden producir comportamientos inesperados dependiendo de la implementación.

Convierte la fila y columna solicitadas a la dirección DDRAM correspondiente del controlador HD44780.

Parámetros

<i>col</i>	Columna destino (0-15).
<i>row</i>	Fila destino (0-1).

6.9. lcd_parallel.h

[Ir a la documentación de este archivo.](#)

```
00001
00035
00036 #pragma once
00037
00038 #include <stdint.h>
00039
00053 void lcd_init(void);
00054
00064 void lcd_clear(void);
00065
00081 void lcd_set_cursor(uint8_t col, uint8_t row);
00082
00095 void lcd_print(const char *str);
```

6.10. Referencia del archivo main/drivers/sd_card.c

Implementación del módulo de almacenamiento en tarjeta SD.

```
#include "sd_card.h"
#include <stdio.h>
#include "esp_err.h"
#include "esp_vfs_fat.h"
#include "sdmmc_cmd.h"
#include "driver/spi_common.h"
#include <errno.h>
#include <sys/stat.h>
```

defines

- **#define PIN_NUM_MISO 19**
GPIO utilizado como línea MISO del bus SPI.
- **#define PIN_NUM_MOSI 23**
GPIO utilizado como línea MOSI del bus SPI.
- **#define PIN_NUM_CLK 18**
GPIO utilizado como reloj SPI.
- **#define PIN_NUM_CS 4**
GPIO utilizado como Chip Select de la SD.
- **#define SD_SPI_MAX_FREQ_KHZ 4000**
Frecuencia máxima del bus SPI en kHz.
- **#define SD_PENDING_MQTT_PATH "/sdcard/test.txt"**
Ruta del archivo utilizado para almacenar mensajes MQTT pendientes.

Funciones

- int `sd_init` (void)
Inicializa la tarjeta SD y monta el sistema de archivos FAT.
- int `sd_write_line` (const char *line)
Escribe una línea en el archivo principal de registro.
- int `sd_append_pending_mqtt` (const char *payload)
Almacena un mensaje MQTT pendiente de transmisión.
- bool `sd_pending_mqtt_exists` (void)
Verifica la existencia del archivo de mensajes pendientes.
- FILE * `sd_open_pending_mqtt_read` (void)
Abre el archivo de mensajes MQTT pendientes.
- char * `sd_read_pending_mqtt_line` (FILE *file, char *buffer, size_t size)
Lee una línea del archivo de mensajes pendientes.
- void `sd_close_pending_mqtt` (FILE *file)
Cierra el archivo de mensajes MQTT pendientes.
- int `sd_delete_pending_mqtt` (void)
Elimina el archivo de mensajes MQTT pendientes.

6.10.1. Descripción detallada

Implementación del módulo de almacenamiento en tarjeta SD.

Este módulo implementa la gestión de una tarjeta SD conectada mediante interfaz SPI utilizando el sistema de archivos FAT proporcionado por ESP-IDF.

Funcionalidades implementadas:

- Inicialización del bus SPI.
- Montaje del sistema de archivos FAT.
- Registro persistente de datos.
- Almacenamiento de mensajes MQTT pendientes.
- Recuperación de mensajes pendientes tras reconexión.

6.10.2. Rol dentro del sistema

Este módulo forma parte del subsistema de persistencia del sistema de fermentación de café.

Se utiliza para:

- Almacenar mediciones ambientales.
- Registrar eventos del sistema.
- Mantener mensajes MQTT durante pérdidas de conectividad.

6.10.3. Estrategia Store-and-Forward

Cuando el broker MQTT no está disponible:

Sensor -> SD

Al recuperarse la conexión:

SD -> MQTT Broker

Esto garantiza que no se pierdan mediciones durante fallos temporales de red.

Nota

El acceso concurrente debe protegerse mediante mutex.

Atención

Escrituras simultáneas sin sincronización pueden provocar corrupción del sistema de archivos FAT.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.10.4. Documentación de «define»

6.10.4.1. PIN_NUM_CLK

```
#define PIN_NUM_CLK 18
```

GPIO utilizado como reloj SPI.

Pin CLK

6.10.4.2. PIN_NUM_CS

```
#define PIN_NUM_CS 4
```

GPIO utilizado como Chip Select de la SD.

Pin Chip Select

6.10.4.3. PIN_NUM_MISO

```
#define PIN_NUM_MISO 19
```

GPIO utilizado como línea MISO del bus SPI.

Pin MISO

6.10.4.4. PIN_NUM_MOSI

```
#define PIN_NUM_MOSI 23
```

GPIO utilizado como línea MOSI del bus SPI.

Pin MOSI

6.10.4.5. SD_PENDING_MQTT_PATH

```
#define SD_PENDING_MQTT_PATH "/sdcard/test.txt"
```

Ruta del archivo utilizado para almacenar mensajes MQTT pendientes.

6.10.4.6. SD_SPI_MAX_FREQ_KHZ

```
#define SD_SPI_MAX_FREQ_KHZ 4000
```

Frecuencia máxima del bus SPI en kHz.

6.10.5. Documentación de funciones

6.10.5.1. sd_append_pending_mqtt()

```
int sd_append_pending_mqtt (  
    const char * payload)
```

Almacena un mensaje MQTT pendiente de transmisión.

Cuando el broker MQTT no está disponible, el payload se guarda en un archivo temporal dentro de la tarjeta SD.

Posteriormente podrá ser reenviado cuando la conectividad sea restaurada.

Parámetros

in	<i>payload</i>	Mensaje MQTT serializado.
----	----------------	---------------------------

Valores devueltos

0	Escritura exitosa.
-1	Error de almacenamiento.

Cuando el sistema detecta que el broker MQTT no está disponible, los mensajes se almacenan temporalmente en un archivo de respaldo dentro de la tarjeta SD.

Una vez restablecida la conexión, dichos mensajes pueden ser reenviados al broker.

Parámetros

in	<i>payload</i>	Mensaje MQTT serializado.
----	----------------	---------------------------

Valores devueltos

0	Escritura exitosa.
-1	Error al almacenar el mensaje.

6.10.5.2. sd_close_pending_mqtt()

```
void sd_close_pending_mqtt (
    FILE * file)
```

Cierra el archivo de mensajes MQTT pendientes.

Libera los recursos asociados al descriptor de archivo.

Parámetros

in	<i>file</i>	Archivo previamente abierto.
----	-------------	------------------------------

6.10.5.3. sd_delete_pending_mqtt()

```
int sd_delete_pending_mqtt (
    void )
```

Elimina el archivo de mensajes MQTT pendientes.

Debe ejecutarse después de retransmitir exitosamente todos los mensajes almacenados.

Valores devueltos

0	Eliminación exitosa.
-1	Error eliminando el archivo.

Esta función debe ejecutarse una vez que todos los mensajes almacenados hayan sido retransmitidos exitosamente al broker.

Valores devueltos

0	Eliminación exitosa o archivo inexistente.
-1	Error durante la eliminación.

6.10.5.4. sd_init()

```
int sd_init (
    void )
```

Inicializa la tarjeta SD y monta el sistema de archivos FAT.

Inicializa y monta la tarjeta SD.

Configura el bus SPI, registra el dispositivo SD y monta el sistema de archivos en la ruta "/sdcard".

Esta función debe ejecutarse antes de cualquier operación de lectura o escritura.

Valores devueltos

0	Inicialización exitosa.
-1	Error de inicialización o montaje.

Nota

Debe ejecutarse una única vez durante el arranque.

Inicializa la tarjeta SD y monta el sistema de archivos FAT.

Configura el bus SPI, registra el dispositivo SD y monta el sistema de archivos FAT en la ruta "/sdcard".

Esta función debe ejecutarse antes de cualquier operación de lectura o escritura.

Valores devueltos

0	Inicialización exitosa.
-1	Error de inicialización o montaje.

Nota

Debe ejecutarse una única vez durante el arranque del sistema.

6.10.5.5. sd_open_pending_mqtt_read()

```
FILE * sd_open_pending_mqtt_read (
    void )
```

Abre el archivo de mensajes MQTT pendientes.

Permite iniciar el proceso de recuperación y retransmisión de mensajes almacenados.

Devuelve

FILE* Descriptor del archivo.

NULL Error al abrir el archivo.

Permite iniciar el proceso de lectura secuencial de los mensajes almacenados durante una pérdida de conectividad.

Devuelve

FILE* Descriptor del archivo abierto.

NULL Error al abrir el archivo.

6.10.5.6. sd_pending_mqtt_exists()

```
bool sd_pending_mqtt_exists (
    void )
```

Verifica la existencia del archivo de mensajes pendientes.

Verifica la existencia del archivo de respaldo MQTT.

Determina si existen datos almacenados para retransmisión MQTT.

Valores devueltos

<i>true</i>	El archivo existe.
<i>false</i>	El archivo no existe.

Verifica la existencia del archivo de mensajes pendientes.

Determina si existen mensajes pendientes almacenados para ser retransmitidos al broker.

Valores devueltos

<i>true</i>	El archivo existe.
<i>false</i>	El archivo no existe.

6.10.5.7. sd_read_pending_mqtt_line()

```
char * sd_read_pending_mqtt_line (
    FILE * file,
    char * buffer,
    size_t size)
```

Lee una línea del archivo de mensajes pendientes.

Lee un mensaje MQTT almacenado en la SD.

Recupera un payload MQTT previamente almacenado en la SD.

Parámetros

in	<i>file</i>	Archivo abierto para lectura.
out	<i>buffer</i>	Buffer de destino.
in	<i>size</i>	Tamaño del buffer.

Devuelve

char* Puntero al buffer si la lectura fue exitosa.
NULL Fin de archivo o error.

Lee una línea del archivo de mensajes pendientes.

Recupera una línea del archivo de respaldo y la almacena en el buffer proporcionado.

Parámetros

in	<i>file</i>	Archivo abierto para lectura.
out	<i>buffer</i>	Buffer de destino.
in	<i>size</i>	Tamaño del buffer.

Devuelve

char* Puntero al buffer si la lectura fue exitosa.
NULL Fin de archivo o error.

6.10.5.8. sd_write_line()

```
int sd_write_line (  
    const char * line)
```

Escribe una línea en el archivo principal de registro.

Añade una línea al archivo principal de registro.

Abre el archivo de log en modo append, añade una nueva línea y cierra el archivo para garantizar la integridad de los datos.

Parámetros

in	<i>line</i>	Cadena de texto a almacenar.
----	-------------	------------------------------

Valores devueltos

0	Escritura exitosa.
-1	Error al abrir el archivo.

Escribe una línea en el archivo principal de registro.

Abre (o crea) el archivo de log y agrega una nueva línea de texto al final del archivo.

Esta función es utilizada para almacenar mediciones, eventos y registros del sistema.

Parámetros

<code>in</code>	<code>line</code>	Cadena de texto a escribir.
-----------------	-------------------	-----------------------------

Valores devueltos

<code>0</code>	Escritura exitosa.
<code>-1</code>	Error de acceso al archivo.

6.11. Referencia del archivo main/drivers/sd_card.h

Módulo de gestión de tarjeta SD mediante interfaz SPI.

```
#include <stddef.h>
#include <stdbool.h>
#include <stdio.h>
```

Funciones

- `int sd_init (void)`
Inicializa y monta la tarjeta SD.
- `int sd_write_line (const char *line)`
Añade una línea al archivo principal de registro.
- `int sd_append_pending_mqtt (const char *payload)`
Almacena un mensaje MQTT pendiente de transmisión.
- `bool sd_pending_mqtt_exists (void)`
Verifica la existencia del archivo de respaldo MQTT.
- `FILE * sd_open_pending_mqtt_read (void)`
Abre el archivo de mensajes MQTT pendientes.
- `char * sd_read_pending_mqtt_line (FILE *file, char *buffer, size_t size)`
Lee un mensaje MQTT almacenado en la SD.
- `void sd_close_pending_mqtt (FILE *file)`
Cierra el archivo de mensajes MQTT pendientes.
- `int sd_delete_pending_mqtt (void)`
Elimina el archivo de mensajes MQTT pendientes.

6.11.1. Descripción detallada

Módulo de gestión de tarjeta SD mediante interfaz SPI.

Este módulo proporciona una capa de abstracción para el acceso a una tarjeta SD utilizando el sistema de archivos FAT y la capa VFS de ESP-IDF.

Funcionalidades principales:

- Inicialización y montaje de la tarjeta SD.
- Registro persistente de datos de sensores.
- Almacenamiento de eventos del sistema.
- Buffer persistente para mensajes MQTT pendientes.

6.11.2. Integración en el sistema

Dentro del sistema de fermentación de café, este módulo cumple dos funciones fundamentales:

- Almacenamiento histórico de mediciones ambientales.
- Persistencia temporal de mensajes MQTT cuando la conexión con el broker no está disponible.

Esta estrategia permite mantener la integridad de los datos incluso ante fallos de red o reinicios inesperados.

Nota

Este módulo no es thread-safe.

El acceso concurrente debe protegerse mediante mutex SPI.

Atención

El acceso simultáneo sin sincronización puede provocar corrupción del sistema de archivos FAT.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.11.3. Documentación de funciones

6.11.3.1. sd_append_pending_mqtt()

```
int sd_append_pending_mqtt (  
    const char * payload)
```

Almacena un mensaje MQTT pendiente de transmisión.

Cuando el sistema detecta que el broker MQTT no está disponible, los mensajes se almacenan temporalmente en un archivo de respaldo dentro de la tarjeta SD.

Una vez restablecida la conexión, dichos mensajes pueden ser reenviados al broker.

Parámetros

in	<i>payload</i>	Mensaje MQTT serializado.
----	----------------	---------------------------

Valores devueltos

0	Escritura exitosa.
---	--------------------

-1	Error al almacenar el mensaje.
----	--------------------------------

Cuando el broker MQTT no está disponible, el payload se guarda en un archivo temporal dentro de la tarjeta SD.

Posteriormente podrá ser reenviado cuando la conectividad sea restaurada.

Parámetros

in	<i>payload</i>	Mensaje MQTT serializado.
----	----------------	---------------------------

Valores devueltos

0	Escritura exitosa.
-1	Error de almacenamiento.

6.11.3.2. `sd_close_pending_mqtt()`

```
void sd_close_pending_mqtt (
    FILE * file)
```

Cierra el archivo de mensajes MQTT pendientes.

Libera los recursos asociados al descriptor de archivo.

Parámetros

in	<i>file</i>	Archivo previamente abierto.
----	-------------	------------------------------

6.11.3.3. `sd_delete_pending_mqtt()`

```
int sd_delete_pending_mqtt (
    void )
```

Elimina el archivo de mensajes MQTT pendientes.

Esta función debe ejecutarse una vez que todos los mensajes almacenados hayan sido retransmitidos exitosamente al broker.

Valores devueltos

0	Eliminación exitosa o archivo inexistente.
-1	Error durante la eliminación.

Debe ejecutarse después de retransmitir exitosamente todos los mensajes almacenados.

Valores devueltos

0	Eliminación exitosa.
-1	Error eliminando el archivo.

6.11.3.4. sd_init()

```
int sd_init (
    void )
```

Inicializa y monta la tarjeta SD.

Inicializa la tarjeta SD y monta el sistema de archivos FAT.

Configura el bus SPI, registra el dispositivo SD y monta el sistema de archivos FAT en la ruta "/sdcard".

Esta función debe ejecutarse antes de cualquier operación de lectura o escritura.

Valores devueltos

0	Inicialización exitosa.
-1	Error de inicialización o montaje.

Nota

Debe ejecutarse una única vez durante el arranque del sistema.

Inicializa y monta la tarjeta SD.

Configura el bus SPI, registra el dispositivo SD y monta el sistema de archivos en la ruta "/sdcard".

Esta función debe ejecutarse antes de cualquier operación de lectura o escritura.

Valores devueltos

0	Inicialización exitosa.
-1	Error de inicialización o montaje.

Nota

Debe ejecutarse una única vez durante el arranque.

6.11.3.5. `sd_open_pending_mqtt_read()`

```
FILE * sd_open_pending_mqtt_read (
    void )
```

Abre el archivo de mensajes MQTT pendientes.

Permite iniciar el proceso de lectura secuencial de los mensajes almacenados durante una pérdida de conectividad.

Devuelve

FILE* Descriptor del archivo abierto.

NULL Error al abrir el archivo.

Permite iniciar el proceso de recuperación y retransmisión de mensajes almacenados.

Devuelve

FILE* Descriptor del archivo.

NULL Error al abrir el archivo.

6.11.3.6. `sd_pending_mqtt_exists()`

```
bool sd_pending_mqtt_exists (
    void )
```

Verifica la existencia del archivo de respaldo MQTT.

Verifica la existencia del archivo de mensajes pendientes.

Determina si existen mensajes pendientes almacenados para ser retransmitidos al broker.

Valores devueltos

<i>true</i>	El archivo existe.
<i>false</i>	El archivo no existe.

Verifica la existencia del archivo de respaldo MQTT.

Determina si existen datos almacenados para retransmisión MQTT.

Valores devueltos

<i>true</i>	El archivo existe.
<i>false</i>	El archivo no existe.

6.11.3.7. sd_read_pending_mqtt_line()

```
char * sd_read_pending_mqtt_line (
    FILE * file,
    char * buffer,
    size_t size)
```

Lee un mensaje MQTT almacenado en la SD.

Lee una línea del archivo de mensajes pendientes.

Recupera una línea del archivo de respaldo y la almacena en el buffer proporcionado.

Parámetros

in	<i>file</i>	Archivo abierto para lectura.
out	<i>buffer</i>	Buffer de destino.
in	<i>size</i>	Tamaño del buffer.

Devuelve

char* Puntero al buffer si la lectura fue exitosa.

NULL Fin de archivo o error.

Lee un mensaje MQTT almacenado en la SD.

Recupera un payload MQTT previamente almacenado en la SD.

Parámetros

in	<i>file</i>	Archivo abierto para lectura.
out	<i>buffer</i>	Buffer de destino.
in	<i>size</i>	Tamaño del buffer.

Devuelve

char* Puntero al buffer si la lectura fue exitosa.

NULL Fin de archivo o error.

6.11.3.8. sd_write_line()

```
int sd_write_line (
    const char * line)
```

Añade una línea al archivo principal de registro.

Escribe una línea en el archivo principal de registro.

Abre (o crea) el archivo de log y agrega una nueva línea de texto al final del archivo.

Esta función es utilizada para almacenar mediciones, eventos y registros del sistema.

Parámetros

in	line	Cadena de texto a escribir.
----	------	-----------------------------

Valores devueltos

0	Escritura exitosa.
-1	Error de acceso al archivo.

Añade una línea al archivo principal de registro.

Abre el archivo de log en modo append, añade una nueva línea y cierra el archivo para garantizar la integridad de los datos.

Parámetros

in	line	Cadena de texto a almacenar.
----	------	------------------------------

Valores devueltos

0	Escritura exitosa.
-1	Error al abrir el archivo.

6.12. sd_card.h

[Ir a la documentación de este archivo.](#)

```

00001
00038
00039 #pragma once
00040
00041 #include <stddef.h>
00042 #include <stdbool.h>
00043 #include <stdio.h>
00044
00060 int sd_init(void);
00061
00076 int sd_write_line(const char *line);
00077
00093 int sd_append_pending_mqtt(const char *payload);
00094
00104 bool sd_pending_mqtt_exists(void);
00105
00115 FILE *sd_open_pending_mqtt_read(void);
00116
00130 char *sd_read_pending_mqtt_line(FILE *file, char *buffer, size_t size);
00131
00139 void sd_close_pending_mqtt(FILE *file);
00140
00150 int sd_delete_pending_mqtt(void);

```

6.13. Referencia del archivo main/include/common.h

Definiciones y estructuras compartidas del sistema.

```
#include <stdint.h>
```


Clases

- struct [datos_ambiente_t](#)

Estructura que almacena una medición ambiental completa.

6.13.1. Descripción detallada

Definiciones y estructuras compartidas del sistema.

Este archivo contiene tipos de datos comunes utilizados por múltiples módulos del ESP32 externo.

Nota

Las estructuras definidas aquí actúan como interfaz de intercambio de información entre tareas y módulos del sistema.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.14. common.h

[Ir a la documentación de este archivo.](#)

```
00001
00016
00017 #pragma once
00018
00019 #include <stdint.h>
00020
00033 typedef struct
00034 {
00040     float temperatura;
00041
00047     float humedad;
00048
00054     uint8_t hora;
00055
00059     uint8_t minuto;
00060
00064     uint8_t segundo;
00065
00069     uint8_t dia;
00070
00074     uint8_t mes;
00075
00079     uint16_t anio;
00080
00081 } datos_ambiente_t;
```

6.15. Referencia del archivo main/include/queues.h

Declaración de las colas FreeRTOS utilizadas por el sistema.

```
#include "freertos/queue.h"
#include "common.h"
```

Variables

- QueueHandle_t [cola_datos](#)
Cola de datos para procesamiento interno.
- QueueHandle_t [cola_display](#)
Cola utilizada para actualización de la interfaz gráfica.
- QueueHandle_t [cola_mqtt](#)
Cola utilizada para transmisión MQTT.

6.15.1. Descripción detallada

Declaración de las colas FreeRTOS utilizadas por el sistema.

Este archivo centraliza las colas empleadas para la comunicación entre tareas del ESP32 externo.

Las colas permiten desacoplar la adquisición de datos, la visualización y la transmisión de información, siguiendo una arquitectura basada en paso de mensajes.

6.15.2. Arquitectura de comunicación

Flujo principal:

task_sensores | +--> cola_datos | +--> cola_display | +--> cola_mqtt

De esta manera las tareas no comparten variables globales directamente, reduciendo problemas de sincronización.

Nota

Las colas son creadas durante la inicialización del sistema y posteriormente utilizadas por las distintas tareas FreeRTOS.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.15.3. Documentación de variables

6.15.3.1. cola_datos

```
QueueHandle_t cola_datos [extern]
```

Cola de datos para procesamiento interno.

Cola utilizada para transferir datos hacia task_logger.

Transporta estructuras de tipo [datos_ambiente_t](#) entre las tareas encargadas de adquisición y procesamiento.

Cola de datos para procesamiento interno.

6.15.3.2. cola_display

```
QueueHandle_t cola_display [extern]
```

Cola utilizada para actualización de la interfaz gráfica.

Cola utilizada para transferir datos hacia task_display.

Permite enviar nuevas mediciones hacia la tarea encargada de controlar la pantalla LCD.

Cola utilizada para actualización de la interfaz gráfica.

6.15.3.3. cola_mqtt

```
QueueHandle_t cola_mqtt [extern]
```

Cola utilizada para transmisión MQTT.

Cola utilizada para transferir datos hacia task_mqtt.

Contiene los datos que posteriormente serán serializados y publicados hacia el broker MQTT.

Cola utilizada para transmisión MQTT.

6.16. queues.h

[Ir a la documentación de este archivo.](#)

```
00001
00035
00036 #pragma once
00037
00038 #include "freertos/queue.h"
00039 #include "common.h"
00040
00047 extern QueueHandle_t cola_datos;
00048
00055 extern QueueHandle_t cola_display;
00056
00063 extern QueueHandle_t cola_mqtt;
```

6.17. Referencia del archivo main/main.c

Punto de entrada del sistema ESP32 para monitoreo ambiental.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/i2c.h"
#include "driver/gpio.h"
#include "esp_sleep.h"
#include "queues.h"
#include "common.h"
#include "ds3231.h"
#include "sd_card.h"
#include "wifi/wifi_manager.h"
```

defines

- #define I2C_MASTER_SCL_IO 22
- #define I2C_MASTER_SDA_IO 21
- #define I2C_MASTER_FREQ_HZ 100000
- #define WIFI_CONNECT_TIMEOUT_MS 10000
- #define RTC_INT_GPIO GPIO_NUM_27

Funciones

- void [task_sensores](#) (void *pvParameters)
Tarea de lectura de sensores.
- void [task_logger](#) (void *pvParameters)
Tarea de registro de datos en SD.
- void [task_display](#) (void *pvParameters)
Tarea de visualización en LCD.
- void [task_mqtt](#) (void *pvParameters)
Tarea responsable de la comunicación MQTT.
- void [i2c_init](#) ()
Inicializa el bus I2C en modo maestro.
- void [app_main](#) (void)
Punto de entrada principal de la aplicación.

Variables

- QueueHandle_t [cola_datos](#)
Cola utilizada para transferir datos hacia task_logger.
- QueueHandle_t [cola_display](#)
Cola utilizada para transferir datos hacia task_display.
- QueueHandle_t [cola_mqtt](#)
Cola utilizada para transferir datos hacia task_mqtt.
- TaskHandle_t [main_task_handle](#) = NULL
Handle de la tarea principal.

6.17.1. Descripción detallada

Punto de entrada del sistema ESP32 para monitoreo ambiental.

Este módulo inicializa el sistema, configura periféricos y coordina la ejecución de tareas bajo FreeRTOS.

Funcionalidades principales:

- Inicialización del bus I2C
- Inicialización de la tarjeta SD
- Creación de colas de comunicación
- Creación de tareas del sistema
- Configuración del RTC DS3231 para wakeup periódico

- Gestión de bajo consumo mediante Deep Sleep

Arquitectura:

- task_sensores → adquisición de datos
- task_logger → almacenamiento en SD
- task_display → visualización en LCD
- task_mqtt → transmisión MQTT

Flujo general:

1. Inicialización del sistema
2. Conexión WiFi
3. Creación de tareas
4. Espera finalización de tareas críticas (display y MQTT)
5. Configuración de alarma RTC
6. Entrada en Deep Sleep

Aspectos de FreeRTOS utilizados:

- Creación dinámica de tareas.
- Colas para intercambio de datos.
- Notificaciones de tareas para sincronización.
- Bloqueo eficiente mediante portMAX_DELAY.

Nota

El sistema funciona en ciclos: medir → procesar → mostrar → guardar → dormir

Atención

El pin SQW del DS3231 debe estar correctamente conectado para permitir el wakeup desde Deep Sleep.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.17.2. Documentación de «define»

6.17.2.1. I2C_MASTER_FREQ_HZ

```
#define I2C_MASTER_FREQ_HZ 100000
```

Frecuencia I2C

6.17.2.2. I2C_MASTER_SCL_IO

```
#define I2C_MASTER_SCL_IO 22
```

Pin SCL

6.17.2.3. I2C_MASTER_SDA_IO

```
#define I2C_MASTER_SDA_IO 21
```

Pin SDA

6.17.2.4. RTC_INT_GPIO

```
#define RTC_INT_GPIO GPIO_NUM_27
```

Pin SQW del DS3231

6.17.2.5. WIFI_CONNECT_TIMEOUT_MS

```
#define WIFI_CONNECT_TIMEOUT_MS 10000
```

6.17.3. Documentación de funciones

6.17.3.1. app_main()

```
void app_main (  
    void )
```

Punto de entrada principal de la aplicación.

Coordina la ejecución completa del ciclo de monitoreo:

- Inicializa periféricos.
- Establece conectividad WiFi.
- Inicializa almacenamiento SD.
- Crea las tareas del sistema.

- Espera la finalización de los procesos críticos.
- Configura la siguiente alarma RTC.
- Entra en modo Deep Sleep.

La función actúa como orquestador del sistema.

Punto de entrada principal de la aplicación.

Coordina la ejecución completa del ciclo de monitoreo:

- Inicializa periféricos.
- Establece conectividad WiFi.
- Inicializa almacenamiento SD.
- Crea las tareas del sistema.
- Espera la finalización de los procesos críticos.
- Configura la siguiente alarma RTC.
- Entra en modo Deep Sleep.

La función actúa como orquestador del sistema. Obtener handle de la tarea principal

Inicializar I2C

Determina la causa de activación desde Deep Sleep

Limpia el flag de alarma del RTC

Evita falsas interrupciones al iniciar el sistema.

Espera la finalización de tareas críticas.

El sistema espera la notificación de:

- task_display
- task_mqtt

antes de continuar hacia el modo de bajo consumo.

Limpia el flag antes de dormir

Retardo para estabilización del pin SQW

Lectura del nivel del pin SQW (debug)

Configura wakeup por señal externa (RTC)

Entrada en Deep Sleep

6.17.3.2. `i2c_init()`

```
void i2c_init ()
```

Inicializa el bus I2C en modo maestro.

Configura los pines SDA y SCL y establece la frecuencia del bus.

Nota

Necesario para comunicación con el RTC DS3231.

6.17.3.3. `task_display()`

```
void task_display (  
    void * pvParameters) [extern]
```

Tarea de visualización en LCD.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados)
---------------------	--

6.17.3.4. `task_logger()`

```
void task_logger (  
    void * pvParameters) [extern]
```

Tarea de registro de datos en SD.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados)
---------------------	--

6.17.3.5. `task_mqtt()`

```
void task_mqtt (  
    void * pvParameters) [extern]
```

Tarea responsable de la comunicación MQTT.

Flujo de ejecución:

1. Espera datos desde la cola MQTT.
2. Construye el mensaje JSON.
3. Inicia el cliente MQTT.
4. Espera conexión con el broker.

5. Reenvía mensajes pendientes.
6. Publica el mensaje actual.
7. Notifica a la tarea principal.
8. Finaliza su ejecución.

En caso de pérdida de conectividad:

- El mensaje actual se almacena en SD.
- La tarea termina sin perder información.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

6.17.3.6. task_sensores()

```
void task_sensores (
    void * pvParameters) [extern]
```

Tarea de lectura de sensores.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados)
---------------------	--

6.17.4. Documentación de variables

6.17.4.1. cola_datos

```
QueueHandle_t cola_datos
```

Cola utilizada para transferir datos hacia task_logger.

Cola de datos para procesamiento interno.

Cola utilizada para transferir datos hacia task_logger.

Transporta estructuras de tipo [datos_ambiente_t](#) entre las tareas encargadas de adquisición y procesamiento.

6.17.4.2. cola_display

```
QueueHandle_t cola_display
```

Cola utilizada para transferir datos hacia task_display.

Cola utilizada para actualización de la interfaz gráfica.

Cola utilizada para transferir datos hacia task_display.

Permite enviar nuevas mediciones hacia la tarea encargada de controlar la pantalla LCD.

6.17.4.3. cola_mqtt

```
QueueHandle_t cola_mqtt
```

Cola utilizada para transferir datos hacia task_mqtt.

Cola utilizada para transmisión MQTT.

Cola utilizada para transferir datos hacia task_mqtt.

Contiene los datos que posteriormente serán serializados y publicados hacia el broker MQTT.

6.17.4.4. main_task_handle

```
TaskHandle_t main_task_handle = NULL
```

Handle de la tarea principal.

Permite sincronización mediante notificaciones desde otras tareas.

Permite notificar al sistema que el ciclo de visualización ha finalizado.

6.18. Referencia del archivo main/mqtt/mqtt_manager.c

Implementación del gestor de comunicación MQTT.

```
#include "mqtt_manager.h"
#include "freertos/FreeRTOS.h"
#include "freertos/event_groups.h"
#include "esp_event.h"
#include "esp_log.h"
#include "mqtt_client.h"
```

defines

- `#define MQTT_BROKER_URI "mqtt://192.168.0.110:1883"`
- `#define MQTT_CONNECTED_BIT BIT0`
Indica que el cliente MQTT está conectado.
- `#define MQTT_PUBLISHED_BIT BIT1`
Indica que una publicación MQTT fue confirmada.
- `#define MQTT_ERROR_BIT BIT2`
Indica que ocurrió un error MQTT.

Funciones

- static void `mqtt_event_handler` (void *handler_args, esp_event_base_t base, int32_t event_id, void *event_data)
Manejador de eventos MQTT.
- void `mqtt_manager_start` (void)
Inicializa y arranca el cliente MQTT.
- bool `mqtt_wait_connected` (uint32_t timeout_ms)
Espera el establecimiento de la conexión MQTT.
- bool `mqtt_publish` (const char *topic, const char *payload, int qos, uint32_t timeout_ms)
Publica un mensaje en un tópico MQTT.
- void `mqtt_manager_stop` (void)
Detiene el cliente MQTT.

Variables

- static const char * `TAG` = "mqtt_manager"
Etiqueta utilizada por ESP_LOG.
- static esp_mqtt_client_handle_t `mqtt_client` = NULL
Instancia del cliente MQTT.
- static EventGroupHandle_t `mqtt_event_group` = NULL
Grupo de eventos utilizado para sincronización MQTT.

6.18.1. Descripción detallada

Implementación del gestor de comunicación MQTT.

Este módulo implementa la conexión con un broker MQTT utilizando la biblioteca ESP-MQTT de ESP-IDF.

La sincronización entre eventos MQTT y tareas de FreeRTOS se realiza mediante Event Groups, permitiendo notificar:

- Estado de conexión.
- Confirmación de publicaciones.
- Errores de comunicación.

El módulo proporciona una interfaz desacoplada para que otras tareas puedan publicar información sin interactuar directamente con el cliente MQTT.

6.18.2. Documentación de «define»

6.18.2.1. MQTT_BROKER_URI

```
#define MQTT_BROKER_URI "mqtt://192.168.0.110:1883"
```

6.18.2.2. MQTT_CONNECTED_BIT

```
#define MQTT_CONNECTED_BIT BIT0
```

Indica que el cliente MQTT está conectado.

6.18.2.3. MQTT_ERROR_BIT

```
#define MQTT_ERROR_BIT BIT2
```

Indica que ocurrió un error MQTT.

6.18.2.4. MQTT_PUBLISHED_BIT

```
#define MQTT_PUBLISHED_BIT BIT1
```

Indica que una publicación MQTT fue confirmada.

6.18.3. Documentación de funciones

6.18.3.1. mqtt_event_handler()

```
void mqtt_event_handler (  
    void * handler_args,  
    esp_event_base_t base,  
    int32_t event_id,  
    void * event_data) [static]
```

Manejador de eventos MQTT.

Recibe los eventos generados por la biblioteca ESP-MQTT y actualiza el grupo de eventos utilizado por FreeRTOS para sincronizar las tareas de aplicación.

Eventos gestionados:

- MQTT_EVENT_CONNECTED
- MQTT_EVENT_DISCONNECTED
- MQTT_EVENT_PUBLISHED
- MQTT_EVENT_ERROR

Parámetros

<i>handler_args</i>	Argumentos del manejador.
<i>base</i>	Base del evento.
<i>event_id</i>	Identificador del evento MQTT.

<i>event_data</i>	Datos asociados al evento.
-------------------	----------------------------

6.18.3.2. mqtt_manager_start()

```
void mqtt_manager_start (  
    void )
```

Inicializa y arranca el cliente MQTT.

Crea el grupo de eventos necesario para la sincronización, configura el cliente MQTT y comienza el proceso de conexión con el broker.

Configura la conexión con el broker MQTT y registra los eventos necesarios para la gestión de la comunicación.

6.18.3.3. mqtt_manager_stop()

```
void mqtt_manager_stop (  
    void )
```

Detiene el cliente MQTT.

Finaliza la conexión con el broker MQTT, destruye la instancia del cliente y limpia los bits de sincronización.

Finaliza la conexión con el broker y libera los recursos asociados al cliente MQTT.

6.18.3.4. mqtt_publish()

```
bool mqtt_publish (  
    const char * topic,  
    const char * payload,  
    int qos,  
    uint32_t timeout_ms)
```

Publica un mensaje en un tópico MQTT.

Para QoS 1 o superior, la función espera la confirmación de publicación utilizando un Event Group.

Parámetros

<i>topic</i>	Tópico MQTT.
<i>payload</i>	Mensaje a publicar.
<i>qos</i>	Nivel de QoS.
<i>timeout_ms</i>	Tiempo máximo de espera.

Devuelve

true si la publicación fue exitosa.
false si ocurrió un error o timeout.

Envía un mensaje al broker utilizando el tópico indicado.

Parámetros

<i>topic</i>	Tópico de destino.
<i>payload</i>	Contenido del mensaje.
<i>qos</i>	Nivel de calidad de servicio MQTT.
<i>timeout_ms</i>	Tiempo máximo de espera para la publicación.

Devuelve

true si la publicación fue exitosa.
false si ocurrió un error o timeout.

6.18.3.5. mqtt_wait_connected()

```
bool mqtt_wait_connected (  
    uint32_t timeout_ms)
```

Espera el establecimiento de la conexión MQTT.

Espera a que el cliente MQTT establezca conexión.

Bloquea la tarea llamadora hasta que se reciba un evento de conexión o se alcance el tiempo máximo de espera.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si se estableció la conexión.
false si ocurrió timeout o error.

Espera el establecimiento de la conexión MQTT.

La función bloquea hasta que se establezca la conexión o hasta que expire el tiempo máximo especificado.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si la conexión fue establecida.
false si ocurrió un timeout.

6.18.4. Documentación de variables

6.18.4.1. mqtt_client

```
esp_mqtt_client_handle_t mqtt_client = NULL [static]
```

Instancia del cliente MQTT.

6.18.4.2. mqtt_event_group

```
EventGroupHandle_t mqtt_event_group = NULL [static]
```

Grupo de eventos utilizado para sincronización MQTT.

6.18.4.3. TAG

```
const char* TAG = "mqtt_manager" [static]
```

Etiqueta utilizada por ESP_LOG.

6.19. Referencia del archivo main/mqtt/mqtt_manager.h

Gestión de la comunicación MQTT del sistema.

```
#include <stdbool.h>
#include <stdint.h>
```

Funciones

- void [mqtt_manager_start](#) (void)
Inicializa y arranca el cliente MQTT.
- bool [mqtt_wait_connected](#) (uint32_t timeout_ms)
Espera a que el cliente MQTT establezca conexión.
- bool [mqtt_publish](#) (const char *topic, const char *payload, int qos, uint32_t timeout_ms)
Publica un mensaje en un tópico MQTT.
- void [mqtt_manager_stop](#) (void)
Detiene el cliente MQTT.

6.19.1. Descripción detallada

Gestión de la comunicación MQTT del sistema.

Este módulo encapsula la inicialización, conexión, publicación y detención del cliente MQTT utilizado por el sistema de monitoreo de fermentación.

Permite que otras tareas de FreeRTOS publiquen datos hacia el broker MQTT sin depender directamente de la implementación interna del protocolo.

6.19.2. Documentación de funciones

6.19.2.1. mqtt_manager_start()

```
void mqtt_manager_start (
    void )
```

Inicializa y arranca el cliente MQTT.

Configura la conexión con el broker MQTT y registra los eventos necesarios para la gestión de la comunicación.

Crea el grupo de eventos necesario para la sincronización, configura el cliente MQTT y comienza el proceso de conexión con el broker.

6.19.2.2. mqtt_manager_stop()

```
void mqtt_manager_stop (
    void )
```

Detiene el cliente MQTT.

Finaliza la conexión con el broker y libera los recursos asociados al cliente MQTT.

Finaliza la conexión con el broker MQTT, destruye la instancia del cliente y limpia los bits de sincronización.

6.19.2.3. mqtt_publish()

```
bool mqtt_publish (
    const char * topic,
    const char * payload,
    int qos,
    uint32_t timeout_ms)
```

Publica un mensaje en un tópico MQTT.

Envía un mensaje al broker utilizando el tópico indicado.

Parámetros

<i>topic</i>	Tópico de destino.
<i>payload</i>	Contenido del mensaje.
<i>qos</i>	Nivel de calidad de servicio MQTT.
<i>timeout_ms</i>	Tiempo máximo de espera para la publicación.

Devuelve

true si la publicación fue exitosa.
false si ocurrió un error o timeout.

Para QoS 1 o superior, la función espera la confirmación de publicación utilizando un Event Group.

Parámetros

<i>topic</i>	Tópico MQTT.
<i>payload</i>	Mensaje a publicar.
<i>qos</i>	Nivel de QoS.
<i>timeout_ms</i>	Tiempo máximo de espera.

Devuelve

true si la publicación fue exitosa.
false si ocurrió un error o timeout.

6.19.2.4. mqtt_wait_connected()

```
bool mqtt_wait_connected (  
    uint32_t timeout_ms)
```

Espera a que el cliente MQTT establezca conexión.

Espera el establecimiento de la conexión MQTT.

La función bloquea hasta que se establezca la conexión o hasta que expire el tiempo máximo especificado.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si la conexión fue establecida.
false si ocurrió un timeout.

Espera a que el cliente MQTT establezca conexión.

Bloquea la tarea llamadora hasta que se reciba un evento de conexión o se alcance el tiempo máximo de espera.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si se estableció la conexión.
false si ocurrió timeout o error.

6.20. mqtt_manager.h

[Ir a la documentación de este archivo.](#)

```
00001
00013
00014 #ifndef MQTT_MANAGER_H
00015 #define MQTT_MANAGER_H
00016
00017 #include <stdbool.h>
00018 #include <stdint.h>
00019
00026 void mqtt_manager_start(void);
00027
00039 bool mqtt_wait_connected(uint32_t timeout_ms);
00040
00054 bool mqtt_publish(const char *topic,
00055                  const char *payload,
00056                  int qos,
00057                  uint32_t timeout_ms);
00058
00065 void mqtt_manager_stop(void);
00066
00067 #endif
```

6.21. Referencia del archivo main/tasks/task_display.c

Tarea encargada de la visualización de datos en LCD.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include <stdio.h>
#include "queues.h"
#include "common.h"
#include "lcd_parallel.h"
```

Funciones

- void `task_display` (void *pvParameters)
Tarea de visualización en LCD.

Variables

- TaskHandle_t `main_task_handle`
Handle de la tarea principal.

6.21.1. Descripción detallada

Tarea encargada de la visualización de datos en LCD.

Esta tarea recibe datos del sistema a través de una cola y los muestra en una pantalla LCD 16x2.

Responsabilidades:

- Formatear datos para visualización

- Actualizar la pantalla LCD
- Sincronizar con la tarea principal mediante notificaciones

Flujo de ejecución:

1. Espera datos desde la cola (bloqueante)
2. Formatea la información (hora, fecha, temperatura, humedad)
3. Actualiza el display
4. Notifica a la tarea principal

Comunicación:

- Entrada: cola_display (Queue)
- Salida: notificación a main_task_handle

Nota

Esta tarea es bloqueante sobre la cola (portMAX_DELAY), lo cual optimiza el consumo energético.

Atención

El acceso al LCD no es thread-safe. Esta tarea debe ser la única que interactúe con el display o debe usarse un mutex.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.21.2. Documentación de funciones

6.21.2.1. task_display()

```
void task_display (  
    void * pvParameters)
```

Tarea de visualización en LCD.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados)
---------------------	--

6.21.3. Documentación de variables

6.21.3.1. main_task_handle

```
TaskHandle_t main_task_handle [extern]
```

Handle de la tarea principal.

Permite notificar al sistema que el ciclo de visualización ha finalizado.

Permite sincronización mediante notificaciones desde otras tareas.

6.22. Referencia del archivo main/tasks/task_logger.c

Tarea encargada del almacenamiento de datos en tarjeta SD.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include <stdio.h>
#include "queues.h"
#include "common.h"
#include "sd_card.h"
```

Funciones

- void [task_logger](#) (void *pvParameters)
Tarea de registro de datos en SD.

6.22.1. Descripción detallada

Tarea encargada del almacenamiento de datos en tarjeta SD.

Esta tarea recibe datos ambientales desde una cola y los almacena en una tarjeta SD en formato CSV.

Responsabilidades:

- Recibir datos desde otras tareas (sensores/control)
- Formatear la información en una línea estructurada
- Guardar los datos en la SD
- Mostrar información de depuración en consola

Flujo de ejecución:

1. Espera datos en la cola (bloqueante)
2. Formatea los datos en formato CSV
3. Imprime en consola (debug)
4. Guarda en tarjeta SD

Comunicación:

- Entrada: cola_datos (Queue)

Nota

Esta tarea es bloqueante sobre la cola, lo que optimiza el uso de CPU y energía.

Atención

El acceso a la SD no es thread-safe. Se recomienda usar un mutex SPI si múltiples tareas acceden al bus.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.22.2. Documentación de funciones**6.22.2.1. task_logger()**

```
void task_logger (
    void * pvParameters)
```

Tarea de registro de datos en SD.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados)
---------------------	--

6.23. Referencia del archivo main/tasks/task_mqtt.c

Tarea encargada de la transmisión MQTT de datos ambientales.

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "common.h"
#include "mqtt/mqtt_manager.h"
#include "queues.h"
#include "sd_card.h"
```

defines

- `#define MQTT_TOPIC "fermentador/espExterior/datos"`
Tópico MQTT utilizado para la publicación.
- `#define MQTT_CONNECT_TIMEOUT_MS 8000`
Tiempo máximo de espera para conexión MQTT.
- `#define MQTT_PUBLISH_TIMEOUT_MS 5000`
Tiempo máximo de espera para confirmación de publicación.
- `#define MQTT_PENDING_PAYLOAD_SIZE 256`
Tamaño máximo del buffer utilizado para mensajes pendientes.

Funciones

- static void [save_pending_payload](#) (const char *payload)
Guarda un payload MQTT pendiente en la tarjeta SD.
- static bool [resend_pending_mqtt_messages](#) (void)
Reenvía mensajes MQTT pendientes almacenados en SD.
- void [task_mqtt](#) (void *pvParameters)
Tarea responsable de la comunicación MQTT.

Variables

- TaskHandle_t [main_task_handle](#)
Handle de la tarea principal.

6.23.1. Descripción detallada

Tarea encargada de la transmisión MQTT de datos ambientales.

Esta tarea recibe datos desde una cola FreeRTOS, genera un mensaje JSON y lo publica en un broker MQTT.

Para aumentar la confiabilidad del sistema, los mensajes que no pueden ser enviados debido a fallos de conectividad son almacenados temporalmente en la tarjeta SD y reenviados automáticamente cuando la conexión se restablece.

Responsabilidades:

- Recibir datos desde la cola MQTT.
- Construir el payload JSON.
- Gestionar la conexión MQTT.
- Publicar datos en el broker.
- Almacenar mensajes pendientes en SD.
- Reenviar mensajes pendientes.
- Notificar la finalización del ciclo.

Aspectos de FreeRTOS utilizados:

- Queue para recepción de datos.
- Task Notification para sincronización.
- Eliminación dinámica de tarea mediante `vTaskDelete()`.

Estrategia de tolerancia a fallos:

- Si MQTT falla, los datos se almacenan en SD.
- Al recuperar la conexión se reenvían los mensajes pendientes.
- Se evita la pérdida de información durante cortes de red.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.23.2. Documentación de «define»

6.23.2.1. MQTT_CONNECT_TIMEOUT_MS

```
#define MQTT_CONNECT_TIMEOUT_MS 8000
```

Tiempo máximo de espera para conexión MQTT.

6.23.2.2. MQTT_PENDING_PAYLOAD_SIZE

```
#define MQTT_PENDING_PAYLOAD_SIZE 256
```

Tamaño máximo del buffer utilizado para mensajes pendientes.

6.23.2.3. MQTT_PUBLISH_TIMEOUT_MS

```
#define MQTT_PUBLISH_TIMEOUT_MS 5000
```

Tiempo máximo de espera para confirmación de publicación.

6.23.2.4. MQTT_TOPIC

```
#define MQTT_TOPIC "fermentador/espExterior/datos"
```

Tópico MQTT utilizado para la publicación.

6.23.3. Documentación de funciones

6.23.3.1. resend_pending_mqtt_messages()

```
bool resend_pending_mqtt_messages (  
    void ) [static]
```

Reenvía mensajes MQTT pendientes almacenados en SD.

Recorre el archivo de mensajes pendientes y publica cada registro en el broker MQTT.

Si todos los mensajes son enviados correctamente, el archivo de pendientes es eliminado.

Devuelve

true si todos los mensajes fueron reenviados.

false si ocurrió algún error durante el proceso.

6.23.3.2. `save_pending_payload()`

```
void save_pending_payload (
    const char * payload) [static]
```

Guarda un payload MQTT pendiente en la tarjeta SD.

Esta función es utilizada cuando no es posible completar una transmisión MQTT debido a problemas de conectividad.

Parámetros

<i>payload</i>	Mensaje MQTT a almacenar.
----------------	---------------------------

6.23.3.3. `task_mqtt()`

```
void task_mqtt (
    void * pvParameters)
```

Tarea responsable de la comunicación MQTT.

Flujo de ejecución:

1. Espera datos desde la cola MQTT.
2. Construye el mensaje JSON.
3. Inicia el cliente MQTT.
4. Espera conexión con el broker.
5. Reenvía mensajes pendientes.
6. Publica el mensaje actual.
7. Notifica a la tarea principal.
8. Finaliza su ejecución.

En caso de pérdida de conectividad:

- El mensaje actual se almacena en SD.
- La tarea termina sin perder información.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

6.23.4. Documentación de variables

6.23.4.1. main_task_handle

```
TaskHandle_t main_task_handle [extern]
```

Handle de la tarea principal.

Utilizado para notificar la finalización del proceso MQTT.

Permite sincronización mediante notificaciones desde otras tareas.

Permite notificar al sistema que el ciclo de visualización ha finalizado.

6.24. Referencia del archivo main/tasks/task_sensores.c

Tarea de adquisición de datos ambientales.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include <stdio.h>
#include "queues.h"
#include "common.h"
#include "dht11.h"
#include "ds3231.h"
```

Funciones

- void `task_sensores` (void *pvParameters)
Tarea de lectura de sensores.

6.24.1. Descripción detallada

Tarea de adquisición de datos ambientales.

Esta tarea realiza una medición única de los sensores del sistema:

- Sensor DHT11 (temperatura y humedad)
- RTC DS3231 (fecha y hora)

Luego:

- Empaqueta los datos en una estructura
- Envía la información a otras tareas mediante colas
- Finaliza su ejecución

Flujo de ejecución:

1. Lectura del sensor DHT11
2. Lectura del RTC
3. Validación de datos
4. Envío a colas:
 - cola_datos → task_logger
 - cola_display → task_display
 - cola_mqtt → task_mqtt
5. Eliminación de la tarea

Comunicación:

- Salida:
 - cola_datos
 - cola_display
 - cola_mqtt

Aspectos de FreeRTOS utilizados:

- Queue para distribución de datos entre tareas.
- Ejecución one-shot mediante creación y destrucción dinámica.
- Bloqueo seguro mediante portMAX_DELAY durante el envío.

Nota

Esta tarea es de tipo "one-shot" (ejecución única).

La periodicidad de ejecución es gestionada por la tarea principal del sistema, manteniendo esta tarea enfocada exclusivamente en la adquisición de datos.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.24.2. Documentación de funciones

6.24.2.1. task_sensores()

```
void task_sensores (
    void * pvParameters)
```

Tarea de lectura de sensores.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados)
---------------------	--

6.25. Referencia del archivo main/wifi/wifi_manager.c

Implementación del gestor de conectividad WiFi.

```
#include "wifi_manager.h"
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/event_groups.h"
#include "freertos/task.h"
#include "esp_event.h"
#include "esp_log.h"
#include "esp_netif.h"
#include "esp_wifi.h"
#include "nvs_flash.h"
```

defines

- #define [WIFI_SSID](#) "Sala331"
- #define [WIFI_PASS](#) "Sala331tm"
- #define [WIFI_MAX_RETRIES](#) 4
- #define [WIFI_SCAN_MAX_AP](#) 20
- #define [WIFI_CONNECTED_BIT](#) BIT0
Bit que indica conexión WiFi exitosa.
- #define [WIFI_FAIL_BIT](#) BIT1
Bit que indica fallo de conexión.

Funciones

- static void [wifi_event_handler](#) (void *arg, esp_event_base_t event_base, int32_t event_id, void *event_data)
Manejador de eventos WiFi e IP.
- void [wifi_init](#) (void)
Inicializa la infraestructura de red WiFi.
- bool [wifi_wait_connected](#) (uint32_t timeout_ms)
Espera el resultado del proceso de conexión WiFi.
- static const char * [auth_mode_name](#) (wifi_auth_mode_t authmode)
Convierte un tipo de autenticación WiFi a texto.
- void [wifi_scan_print](#) (void)
Escanea e imprime las redes WiFi visibles.

Variables

- static const char * TAG = "wifi_manager"
Etiqueta utilizada por el sistema de logs.
- static int retry_count = 0
Contador de reintentos de conexión WiFi.
- static EventGroupHandle_t wifi_event_group
Grupo de eventos utilizado para sincronizar el estado de la conexión WiFi.

6.25.1. Descripción detallada

Implementación del gestor de conectividad WiFi.

Este módulo implementa la inicialización y administración de la interfaz WiFi del ESP32 utilizando ESP-IDF.

Funcionalidades:

- Inicialización de NVS.
- Configuración del stack TCP/IP.
- Gestión de conexión WiFi en modo estación.
- Reintentos automáticos de conexión.
- Obtención de dirección IP.
- Escaneo de redes disponibles.

Aspectos de FreeRTOS utilizados:

- Event Groups para sincronización de estados.
- Tareas bloqueantes mediante xEventGroupWaitBits().

La comunicación entre los eventos del stack WiFi y las tareas de aplicación se realiza mediante bits de estado almacenados en un Event Group.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.25.2. Documentación de «define»

6.25.2.1. WIFI_CONNECTED_BIT

```
#define WIFI_CONNECTED_BIT BIT0
```

Bit que indica conexión WiFi exitosa.

6.25.2.2. WIFI_FAIL_BIT

```
#define WIFI_FAIL_BIT BIT1
```

Bit que indica fallo de conexión.

6.25.2.3. WIFI_MAX_RETRIES

```
#define WIFI_MAX_RETRIES 4
```

6.25.2.4. WIFI_PASS

```
#define WIFI_PASS "Sala331tm"
```

6.25.2.5. WIFI_SCAN_MAX_AP

```
#define WIFI_SCAN_MAX_AP 20
```

6.25.2.6. WIFI_SSID

```
#define WIFI_SSID "Sala331"
```

6.25.3. Documentación de funciones

6.25.3.1. auth_mode_name()

```
const char * auth_mode_name (  
    wifi_auth_mode_t authmode) [static]
```

Convierte un tipo de autenticación WiFi a texto.

Parámetros

<i>authmode</i>	Modo de autenticación.
-----------------	------------------------

Devuelve

Cadena descriptiva del modo de autenticación.

6.25.3.2. `wifi_event_handler()`

```
void wifi_event_handler (
    void * arg,
    esp_event_base_t event_base,
    int32_t event_id,
    void * event_data) [static]
```

Manejador de eventos WiFi e IP.

Procesa los eventos generados por el stack de red de ESP-IDF y actualiza el Event Group utilizado por las tareas del sistema.

Eventos gestionados:

- `WIFI_EVENT_STA_START`
- `WIFI_EVENT_STA_DISCONNECTED`
- `IP_EVENT_STA_GOT_IP`

Parámetros

<i>arg</i>	Argumento de usuario.
<i>event_base</i>	Base del evento.
<i>event_id</i>	Identificador del evento.
<i>event_data</i>	Datos asociados al evento.

6.25.3.3. `wifi_init()`

```
void wifi_init (
    void )
```

Inicializa la infraestructura de red WiFi.

Inicializa y conecta el ESP32 a una red WiFi.

Realiza las siguientes operaciones:

- Inicialización de NVS.
- Inicialización del stack TCP/IP.
- Creación del Event Loop.
- Configuración del modo estación.
- Registro de eventos WiFi.
- Inicio de la interfaz inalámbrica.

La conexión al punto de acceso comienza automáticamente después de la inicialización.

Inicializa la infraestructura de red WiFi.

Configura los recursos necesarios de red e inicia el proceso de asociación con el punto de acceso. Crear event group.

Inicializar NVS.

Inicializar TCP/IP stack.

Crear event loop.

Crear interfaz WiFi station.

Configuración WiFi por defecto.

Registrar handlers de eventos.

Configuración de credenciales WiFi.

Configurar modo station.

Aplicar configuración WiFi.

Iniciar WiFi.

6.25.3.4. wifi_scan_print()

```
void wifi_scan_print (
    void )
```

Escanea e imprime las redes WiFi visibles.

Escanea las redes WiFi disponibles.

Realiza un escaneo activo de puntos de acceso cercanos e imprime información relevante:

- SSID
- Intensidad de señal (RSSI)
- Canal
- Tipo de autenticación

Esta función está destinada principalmente a tareas de diagnóstico y depuración.

Escanea e imprime las redes WiFi visibles.

Realiza un barrido de redes inalámbricas visibles e imprime la información obtenida mediante la consola de depuración.

6.25.3.5. `wifi_wait_connected()`

```
bool wifi_wait_connected (
    uint32_t timeout_ms)
```

Espera el resultado del proceso de conexión WiFi.

Espera hasta que se establezca la conexión WiFi.

La función permanece bloqueada hasta que:

- Se obtiene una dirección IP válida.
- Se alcanza el número máximo de reintentos.
- Expira el timeout especificado.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si se estableció la conexión.
false si ocurrió timeout o fallo.

Espera el resultado del proceso de conexión WiFi.

La función bloquea hasta que el dispositivo obtenga una dirección IP válida o se alcance el tiempo máximo de espera.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si la conexión fue exitosa.
false si ocurrió timeout o error.

6.25.4. Documentación de variables

6.25.4.1. `retry_count`

```
int retry_count = 0 [static]
```

Contador de reintentos de conexión WiFi.

6.25.4.2. TAG

```
const char* TAG = "wifi_manager" [static]
```

Etiqueta utilizada por el sistema de logs.

6.25.4.3. wifi_event_group

```
EventGroupHandle_t wifi_event_group [static]
```

Grupo de eventos utilizado para sincronizar el estado de la conexión WiFi.

Permite notificar a otras tareas cuando:

- Se establece la conexión.
- Ocurre un fallo de conexión.

6.26. Referencia del archivo main/wifi/wifi_manager.h

Gestión de conectividad WiFi para el ESP32.

```
#include <stdbool.h>
#include <stdint.h>
```

Funciones

- void `wifi_init` (void)
Inicializa y conecta el ESP32 a una red WiFi.
- bool `wifi_wait_connected` (uint32_t timeout_ms)
Espera hasta que se establezca la conexión WiFi.
- void `wifi_scan_print` (void)
Escanea las redes WiFi disponibles.

6.26.1. Descripción detallada

Gestión de conectividad WiFi para el ESP32.

Este módulo proporciona una interfaz para la inicialización, conexión y monitoreo de la red WiFi utilizada por el sistema.

Responsabilidades:

- Inicializar la pila TCP/IP.
- Configurar la interfaz WiFi.
- Gestionar la conexión a la red inalámbrica.
- Verificar el estado de conexión.
- Realizar escaneo de redes disponibles.

Este módulo abstrae los detalles de la implementación WiFi, permitiendo que otras tareas del sistema utilicen servicios de red sin depender directamente de ESP-IDF.

Autores

- Fernando Plazas
- Isabella Ordoñez
- Juan Daniel Constain

6.26.2. Documentación de funciones

6.26.2.1. wifi_init()

```
void wifi_init (
    void )
```

Inicializa y conecta el ESP32 a una red WiFi.

Inicializa la infraestructura de red WiFi.

Configura los recursos necesarios de red e inicia el proceso de asociación con el punto de acceso.

Inicializa y conecta el ESP32 a una red WiFi.

Realiza las siguientes operaciones:

- Inicialización de NVS.
- Inicialización del stack TCP/IP.
- Creación del Event Loop.
- Configuración del modo estación.
- Registro de eventos WiFi.
- Inicio de la interfaz inalámbrica.

La conexión al punto de acceso comienza automáticamente después de la inicialización. Crear event group.

Inicializar NVS.

Inicializar TCP/IP stack.

Crear event loop.

Crear interfaz WiFi station.

Configuración WiFi por defecto.

Registrar handlers de eventos.

Configuración de credenciales WiFi.

Configurar modo station.

Aplicar configuración WiFi.

Iniciar WiFi.

6.26.2.2. wifi_scan_print()

```
void wifi_scan_print (
    void )
```

Escanea las redes WiFi disponibles.

Escanea e imprime las redes WiFi visibles.

Realiza un barrido de redes inalámbricas visibles e imprime la información obtenida mediante la consola de depuración.

Escanea las redes WiFi disponibles.

Realiza un escaneo activo de puntos de acceso cercanos e imprime información relevante:

- SSID
- Intensidad de señal (RSSI)
- Canal
- Tipo de autenticación

Esta función está destinada principalmente a tareas de diagnóstico y depuración.

6.26.2.3. wifi_wait_connected()

```
bool wifi_wait_connected (
    uint32_t timeout_ms)
```

Espera hasta que se establezca la conexión WiFi.

Espera el resultado del proceso de conexión WiFi.

La función bloquea hasta que el dispositivo obtenga una dirección IP válida o se alcance el tiempo máximo de espera.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si la conexión fue exitosa.
false si ocurrió timeout o error.

Espera hasta que se establezca la conexión WiFi.

La función permanece bloqueada hasta que:

- Se obtiene una dirección IP válida.
- Se alcanza el número máximo de reintentos.
- Expira el timeout especificado.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si se estableció la conexión.
false si ocurrió timeout o fallo.

6.27. wifi_manager.h

[Ir a la documentación de este archivo.](#)

```
00001
00024
00025 #ifndef WIFI_MANAGER_H
00026 #define WIFI_MANAGER_H
00027
00028 #include <stdbool.h>
00029 #include <stdint.h>
00030
00037 void wifi_init(void);
00038
00050 bool wifi_wait_connected(uint32_t timeout_ms);
00051
00058 void wifi_scan_print(void);
00059
00060 #endif
```

Índice alfabético

anio
 datos_ambiente_t, [10](#)
 rtc_time_t, [12](#)
app_main
 main.c, [62](#)
auth_mode_name
 wifi_manager.c, [85](#)

bcd_to_dec
 ds3231.c, [22](#)

cola_datos
 main.c, [65](#)
 queues.h, [58](#)
cola_display
 main.c, [65](#)
 queues.h, [58](#)
cola_mqtt
 main.c, [65](#)
 queues.h, [59](#)

datos_ambiente_t, [9](#)
 anio, [10](#)
 dia, [10](#)
 hora, [10](#)
 humedad, [10](#)
 mes, [10](#)
 minuto, [10](#)
 segundo, [10](#)
 temperatura, [10](#)

dec_to_bcd
 ds3231.c, [22](#)

dht11.c
 DHT11_GPIO, [16](#)
 dht11_read, [16](#)
 set_input, [17](#)
 set_output, [17](#)
dht11.h
 dht11_read, [19](#)
dht11_data_t, [11](#)
 humedad, [11](#)
 temperatura, [11](#)
DHT11_GPIO
 dht11.c, [16](#)
dht11_read
 dht11.c, [16](#)
 dht11.h, [19](#)
dia
 datos_ambiente_t, [10](#)
 rtc_time_t, [12](#)

ds3231.c
 bcd_to_dec, [22](#)
 dec_to_bcd, [22](#)
 DS3231_ADDR, [21](#)
 ds3231_clear_alarm_flag, [23](#)
 ds3231_read_time, [24](#)
 ds3231_set_alarm_every_minute, [25](#)
 ds3231_set_time, [26](#)
 I2C_PORT, [21](#)
 TAG, [21](#)
ds3231.h
 ds3231_clear_alarm_flag, [28](#)
 ds3231_read_time, [29](#)
 ds3231_set_alarm_every_minute, [30](#)
 ds3231_set_time, [31](#)
DS3231_ADDR
 ds3231.c, [21](#)
ds3231_clear_alarm_flag
 ds3231.c, [23](#)
 ds3231.h, [28](#)
ds3231_read_time
 ds3231.c, [24](#)
 ds3231.h, [29](#)
ds3231_set_alarm_every_minute
 ds3231.c, [25](#)
 ds3231.h, [30](#)
ds3231_set_time
 ds3231.c, [26](#)
 ds3231.h, [31](#)

hora
 datos_ambiente_t, [10](#)
horas
 rtc_time_t, [12](#)
humedad
 datos_ambiente_t, [10](#)
 dht11_data_t, [11](#)

i2c_init
 main.c, [63](#)
I2C_MASTER_FREQ_HZ
 main.c, [62](#)
I2C_MASTER_SCL_IO
 main.c, [62](#)
I2C_MASTER_SDA_IO
 main.c, [62](#)
I2C_PORT
 ds3231.c, [21](#)

lcd_clear

- lcd_parallel.c, 35
 - lcd_parallel.h, 40
- lcd_cmd
 - lcd_parallel.c, 35
- LCD_D4
 - lcd_parallel.c, 34
- LCD_D5
 - lcd_parallel.c, 34
- LCD_D6
 - lcd_parallel.c, 34
- LCD_D7
 - lcd_parallel.c, 35
- lcd_data
 - lcd_parallel.c, 36
- LCD_E
 - lcd_parallel.c, 35
- lcd_init
 - lcd_parallel.c, 36
 - lcd_parallel.h, 40
- lcd_parallel.c
 - lcd_clear, 35
 - lcd_cmd, 35
 - LCD_D4, 34
 - LCD_D5, 34
 - LCD_D6, 34
 - LCD_D7, 35
 - lcd_data, 36
 - LCD_E, 35
 - lcd_init, 36
 - lcd_print, 36
 - LCD_RS, 35
 - lcd_send_byte, 37
 - lcd_send_nibble, 38
 - lcd_set_cursor, 38
- lcd_parallel.h
 - lcd_clear, 40
 - lcd_init, 40
 - lcd_print, 40
 - lcd_set_cursor, 41
- lcd_print
 - lcd_parallel.c, 36
 - lcd_parallel.h, 40
- LCD_RS
 - lcd_parallel.c, 35
- lcd_send_byte
 - lcd_parallel.c, 37
- lcd_send_nibble
 - lcd_parallel.c, 38
- lcd_set_cursor
 - lcd_parallel.c, 38
 - lcd_parallel.h, 41
- main.c
 - app_main, 62
 - cola_datos, 65
 - cola_display, 65
 - cola_mqtt, 65
 - i2c_init, 63
 - I2C_MASTER_FREQ_HZ, 62
 - I2C_MASTER_SCL_IO, 62
 - I2C_MASTER_SDA_IO, 62
 - main_task_handle, 66
 - RTC_INT_GPIO, 62
 - task_display, 64
 - task_logger, 64
 - task_mqtt, 64
 - task_sensores, 65
 - WIFI_CONNECT_TIMEOUT_MS, 62
- main/drivers/dht11.c, 15
- main/drivers/dht11.h, 18, 20
- main/drivers/ds3231.c, 20
- main/drivers/ds3231.h, 27, 32
- main/drivers/lcd_parallel.c, 33
- main/drivers/lcd_parallel.h, 39, 42
- main/drivers/sd_card.c, 42
- main/drivers/sd_card.h, 50, 56
- main/include/common.h, 56, 57
- main/include/queues.h, 57, 59
- main/main.c, 59
- main/mqtt/mqtt_manager.c, 66
- main/mqtt/mqtt_manager.h, 71, 74
- main/tasks/task_display.c, 74
- main/tasks/task_logger.c, 76
- main/tasks/task_mqtt.c, 77
- main/tasks/task_sensores.c, 81
- main/wifi/wifi_manager.c, 83
- main/wifi/wifi_manager.h, 89, 92
- main_task_handle
 - main.c, 66
 - task_display.c, 76
 - task_mqtt.c, 81
- mes
 - datos_ambiente_t, 10
 - rtc_time_t, 13
- minuto
 - datos_ambiente_t, 10
- minutos
 - rtc_time_t, 13
- MQTT_BROKER_URI
 - mqtt_manager.c, 67
- mqtt_client
 - mqtt_manager.c, 71
- MQTT_CONNECT_TIMEOUT_MS
 - task_mqtt.c, 79
- MQTT_CONNECTED_BIT
 - mqtt_manager.c, 67
- MQTT_ERROR_BIT
 - mqtt_manager.c, 68
- mqtt_event_group
 - mqtt_manager.c, 71
- mqtt_event_handler
 - mqtt_manager.c, 68
- mqtt_manager.c
 - MQTT_BROKER_URI, 67
 - mqtt_client, 71
 - MQTT_CONNECTED_BIT, 67
 - MQTT_ERROR_BIT, 68

- mqtt_event_group, 71
- mqtt_event_handler, 68
- mqtt_manager_start, 69
- mqtt_manager_stop, 69
- mqtt_publish, 69
- MQTT_PUBLISHED_BIT, 68
- mqtt_wait_connected, 70
- TAG, 71
- mqtt_manager.h
 - mqtt_manager_start, 72
 - mqtt_manager_stop, 72
 - mqtt_publish, 72
 - mqtt_wait_connected, 73
- mqtt_manager_start
 - mqtt_manager.c, 69
 - mqtt_manager.h, 72
- mqtt_manager_stop
 - mqtt_manager.c, 69
 - mqtt_manager.h, 72
- MQTT_PENDING_PAYLOAD_SIZE
 - task_mqtt.c, 79
- mqtt_publish
 - mqtt_manager.c, 69
 - mqtt_manager.h, 72
- MQTT_PUBLISH_TIMEOUT_MS
 - task_mqtt.c, 79
- MQTT_PUBLISHED_BIT
 - mqtt_manager.c, 68
- MQTT_TOPIC
 - task_mqtt.c, 79
- mqtt_wait_connected
 - mqtt_manager.c, 70
 - mqtt_manager.h, 73
- PIN_NUM_CLK
 - sd_card.c, 44
- PIN_NUM_CS
 - sd_card.c, 44
- PIN_NUM_MISO
 - sd_card.c, 44
- PIN_NUM_MOSI
 - sd_card.c, 44
- queues.h
 - cola_datos, 58
 - cola_display, 58
 - cola_mqtt, 59
- Referencia del directorio main, 8
- Referencia del directorio main/drivers, 7
- Referencia del directorio main/include, 7
- Referencia del directorio main/mqtt, 8
- Referencia del directorio main/tasks, 8
- Referencia del directorio main/wifi, 8
- resend_pending_mqtt_messages
 - task_mqtt.c, 79
- retry_count
 - wifi_manager.c, 88
- RTC_INT_GPIO
 - main.c, 62
- rtc_time_t, 12
 - anio, 12
 - dia, 12
 - horas, 12
 - mes, 13
 - minutos, 13
 - segundos, 13
- save_pending_payload
 - task_mqtt.c, 79
- sd_append_pending_mqtt
 - sd_card.c, 45
 - sd_card.h, 51
- sd_card.c
 - PIN_NUM_CLK, 44
 - PIN_NUM_CS, 44
 - PIN_NUM_MISO, 44
 - PIN_NUM_MOSI, 44
 - sd_append_pending_mqtt, 45
 - sd_close_pending_mqtt, 46
 - sd_delete_pending_mqtt, 46
 - sd_init, 46
 - sd_open_pending_mqtt_read, 47
 - sd_pending_mqtt_exists, 47
 - SD_PENDING_MQTT_PATH, 45
 - sd_read_pending_mqtt_line, 48
 - SD_SPI_MAX_FREQ_KHZ, 45
 - sd_write_line, 49
- sd_card.h
 - sd_append_pending_mqtt, 51
 - sd_close_pending_mqtt, 52
 - sd_delete_pending_mqtt, 52
 - sd_init, 53
 - sd_open_pending_mqtt_read, 53
 - sd_pending_mqtt_exists, 54
 - sd_read_pending_mqtt_line, 54
 - sd_write_line, 55
- sd_close_pending_mqtt
 - sd_card.c, 46
 - sd_card.h, 52
- sd_delete_pending_mqtt
 - sd_card.c, 46
 - sd_card.h, 52
- sd_init
 - sd_card.c, 46
 - sd_card.h, 53
- sd_open_pending_mqtt_read
 - sd_card.c, 47
 - sd_card.h, 53
- sd_pending_mqtt_exists
 - sd_card.c, 47
 - sd_card.h, 54
- SD_PENDING_MQTT_PATH
 - sd_card.c, 45
- sd_read_pending_mqtt_line
 - sd_card.c, 48
 - sd_card.h, 54
- SD_SPI_MAX_FREQ_KHZ

- sd_card.c, 45
- sd_write_line
 - sd_card.c, 49
 - sd_card.h, 55
- segundo
 - datos_ambiente_t, 10
- segundos
 - rtc_time_t, 13
- set_input
 - dht11.c, 17
- set_output
 - dht11.c, 17
- TAG
 - ds3231.c, 21
 - mqtt_manager.c, 71
 - wifi_manager.c, 88
- task_display
 - main.c, 64
 - task_display.c, 75
- task_display.c
 - main_task_handle, 76
 - task_display, 75
- task_logger
 - main.c, 64
 - task_logger.c, 77
- task_logger.c
 - task_logger, 77
- task_mqtt
 - main.c, 64
 - task_mqtt.c, 80
- task_mqtt.c
 - main_task_handle, 81
 - MQTT_CONNECT_TIMEOUT_MS, 79
 - MQTT_PENDING_PAYLOAD_SIZE, 79
 - MQTT_PUBLISH_TIMEOUT_MS, 79
 - MQTT_TOPIC, 79
 - resend_pending_mqtt_messages, 79
 - save_pending_payload, 79
 - task_mqtt, 80
- task_sensores
 - main.c, 65
 - task_sensores.c, 83
- task_sensores.c
 - task_sensores, 83
- temperatura
 - datos_ambiente_t, 10
 - dht11_data_t, 11
- WIFI_CONNECT_TIMEOUT_MS
 - main.c, 62
- WIFI_CONNECTED_BIT
 - wifi_manager.c, 84
- wifi_event_group
 - wifi_manager.c, 89
- wifi_event_handler
 - wifi_manager.c, 85
- WIFI_FAIL_BIT
 - wifi_manager.c, 84
- wifi_init
 - wifi_manager.c, 86
 - wifi_manager.h, 90
- wifi_manager.c
 - auth_mode_name, 85
 - retry_count, 88
 - TAG, 88
 - WIFI_CONNECTED_BIT, 84
 - wifi_event_group, 89
 - wifi_event_handler, 85
 - WIFI_FAIL_BIT, 84
 - wifi_init, 86
 - WIFI_MAX_RETRIES, 85
 - WIFI_PASS, 85
 - WIFI_SCAN_MAX_AP, 85
 - wifi_scan_print, 87
 - WIFI_SSID, 85
 - wifi_wait_connected, 87
- wifi_manager.h
 - wifi_init, 90
 - wifi_scan_print, 90
 - wifi_wait_connected, 91
- WIFI_MAX_RETRIES
 - wifi_manager.c, 85
- WIFI_PASS
 - wifi_manager.c, 85
- WIFI_SCAN_MAX_AP
 - wifi_manager.c, 85
- wifi_scan_print
 - wifi_manager.c, 87
 - wifi_manager.h, 90
- WIFI_SSID
 - wifi_manager.c, 85
- wifi_wait_connected
 - wifi_manager.c, 87
 - wifi_manager.h, 91